FakeScope: AI-Integrated Fake Review Detection System for E-Commerce Platforms

UBCA398J – Project

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Computer Applications

by

RYAN SANGMA

23BCA0XXX

Under the guidance of

Prof. / Dr. [Guide Name]

[Designation], SCORE, VIT, Vellore

School of Computer Science Engineering and Information Systems

VIT, Vellore

April, 2026

DECLARATION

I hereby declare that the UBCA398J – Project entitled "FakeScope: AI-Integrated Fake Review Detection System for E-Commerce Platforms" submitted by me, for the award of the degree of Bachelor of Computer Applications, School of Computer Science Engineering and Information Systems to VIT is a record of bonafide work carried out by me under the supervision of [Guide Name, Designation, SCORE], VIT, Vellore.

I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Vellore

Date:

Signature of the Candidate

CERTIFICATE

This is to certify that the UBCA398J – Project entitled "FakeScope: AI-Integrated Fake Review Detection System for E-Commerce Platforms" submitted by RYAN SANGMA (23BCA0XXX), SCORE, VIT, for the award of the degree of Bachelor of Computer Applications, School of Computer Science Engineering and Information Systems, is a record of bonafide work carried out by him under my supervision during the period, 05. 12. 2025 to 17. 04. 2026, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The thesis fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place: Vellore

Date:

Signature of the Guide

Internal Examiner External Examiner

Head of the Department

Department of Computer Applications

ACKNOWLEDGEMENT

It is my pleasure to express with a deep sense of gratitude to my UBCA398J – Project guide [Guide Name, Designation], School of Computer Science Engineering and Information Systems, Vellore Institute of Technology, Vellore, for constant guidance, continual encouragement, and understanding throughout the duration of this project. My association with them is not confined to academics only, but it has been a great opportunity to work with an intellectual and expert in the field of Machine Learning and Natural Language Processing.

I would like to express my heartfelt gratitude to Honorable Chancellor Dr. G Viswanathan; respected Vice Presidents Mr. Sankar Viswanathan, Dr. Sekar Viswanathan; Vice Chancellor Dr. V. S. Kanchana Bhaaskaran; Pro-Vice Chancellor Dr. Partha Sharathi Mallick; and Registrar Dr. Jayabarathi T.

My whole-hearted thanks to Dean Dr. Daphne Lopez, School of Computer Science Engineering and Information Systems; Head, Department of Computer Applications, Dr. Vijayan E; Project Coordinator Dr. Ephzibah E.P; SCORE School Project Coordinator Dr. Thandeeswaran R; and all faculty, staff and members working as limbs of our university for their continuous guidance throughout my course of study.

I am grateful to the open-source community behind Python, Scikit-learn, Streamlit, Groq, and the Hugging Face ecosystem, whose freely available tools made this research possible. I also acknowledge the creators of the Yelp Academic Dataset and the Amazon Product Review Dataset for making their data publicly available for research.

It is indeed a pleasure to thank my parents and friends who persuaded and encouraged me to take up and complete this project successfully. Last but not least, I express my gratitude to all those who helped me directly or indirectly towards the successful completion of this project.

Place: Vellore RYAN SANGMA

Date: Name of the Student

EXECUTIVE SUMMARY

FakeScope is an AI-integrated fake review detection system designed to help online shoppers identify deceptive product reviews on e-commerce platforms. The system employs a three-layer hybrid detection pipeline: a rule-based filter for instant flagging of short, overly positive reviews; a Hybrid Machine Learning model combining TF-IDF text vectorization with six stylometric features and Logistic Regression; and an agentic Large Language Model layer powered by Groq and Llama 3.1 that generates natural language buying verdicts. Trained on a balanced dataset of 50,358 reviews from Yelp and Amazon, the pipeline achieves 87.2% accuracy and a ROC-AUC of 0.951 — outperforming BERT zero-shot by 38 percentage points and all classical baselines. The interactive Streamlit dashboard provides four tabs: Quick Check for instant single-review analysis, HTML Upload for batch analysis of saved Amazon pages with behavioral signal parsing, AI Summary for Groq-powered buying verdicts, and Model Metrics for full academic evaluation. The system computes a Product Trust Score (0–100) incorporating fake rate, sentiment, aspect analysis, and behavioral signals extracted from review metadata.

Keywords: Fake Review Detection, TF-IDF, Stylometric Features, Logistic Regression, Groq API, Llama 3.1, Streamlit, Trust Score, Sentiment Analysis, Natural Language Processing.

TABLE OF CONTENTS

No.	Title	Page No.
	Acknowledgement	iv
	Executive Summary	v
	Table of Contents	vi
	List of Figures	vii
	List of Tables	viii
	Abbreviations	ix
	Symbols and Notations	x
1	Introduction	1
	1.1 Objective	2
	1.2 Motivation	3
	1.3 Background	4
2	Project Description and Goals	6
	2.1 Project Description	6
	2.2 Goals	8
	2.3 Research Gap	9
	2.4 Problem Statement	10
	2.5 Literature Survey	11
3	Technical Specification	16
	3.1 Existing System	16
	3.2 Proposed System	18
	3.3 Module Description	19
	3.4 Algorithms	23
	3.5 Software & Hardware Requirements	27
4	Design Approach and Details	29
	4.1 System Architecture	29

No.	Title	Page No.
	4.2 Data Flow Diagram	31
	4.3 Use Case Diagram	33
	4.4 Sequence Diagram	34
	4.5 Activity Diagram	35
	4.6 ER Diagram	36
	4.7 Constraints and Tradeoffs	37
5	Schedule, Tasks and Milestones	39
6	Implementation and Demonstration	42
	6.1 Environment Setup	42
	6.2 System Configuration	44
	6.3 Working Explanation	45
	6.4 Module Outputs Description	48
7	Results and Discussion	51
8	Conclusion	55
9	Future Scope	56
10	Summary	58
11	References	59
12	Appendix – Source Code	62

LIST OF FIGURES

Figure No.	Title	Page No.
Fig. 4.1	System Architecture Diagram	29
Fig. 4.2	Data Flow Diagram – Level 0	31
Fig. 4.3	Data Flow Diagram – Level 1	32
Fig. 4.4	Use Case Diagram	33
Fig. 4.5	Sequence Diagram – Quick Check Flow	34
Fig. 4.6	Activity Diagram – HTML Upload Analysis	35
Fig. 4.7	ER Diagram – Review Analysis Data Model	36
Fig. 6.1	Quick Check Tab – Single Review Input	48
Fig. 6.2	HTML Upload Tab – Batch Analysis Output	49
Fig. 6.3	AI Summary Tab – Groq Buying Verdict	50
Fig. 6.4	Model Metrics Tab – Baseline Comparison	51
Fig. 7.1	5-Fold Cross-Validation Accuracy Chart	52
Fig. 7.2	ROC Curves – All Baseline Models	53
Fig. 7.3	Confusion Matrix Heatmap	53
Fig. 7.4	Ablation Study – Feature Contribution	54

LIST OF TABLES

Table No.	Title	Page No.
Table 2.1	Literature Survey	11
Table 3.1	Existing vs. Proposed System Comparison	18
Table 3.2	Module Summary	22
Table 3.3	Software Requirements	27
Table 3.4	Hardware Requirements	28
Table 4.1	Constraints and Tradeoffs	37
Table 5.1	Project Schedule and Milestones	39
Table 7.1	Baseline Model Comparison	51
Table 7.2	Ablation Study Results	54
Table 7.3	Confusion Matrix Values	53
Table 7.4	Trust Score Grade Scale	54

ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
CV	Cross-Validation
DFD	Data Flow Diagram
EDA	Exploratory Data Analysis
F1	F1-Score (Harmonic Mean of Precision and Recall)
FN	False Negative
FP	False Positive
FPR	False Positive Rate
HTML	HyperText Markup Language
LLM	Large Language Model
LR	Logistic Regression
ML	Machine Learning
NLP	Natural Language Processing
ROC-AUC	Receiver Operating Characteristic – Area Under the Curve
RF	Random Forest
SVM	Support Vector Machine
TF-IDF	Term Frequency – Inverse Document Frequency
TN	True Negative
TP	True Positive
TPR	True Positive Rate
VADER	Valence Aware Dictionary and sEntiment Reasoner

SYMBOLS AND NOTATIONS

Symbol	Meaning
→	Implies or directs to
%	Percentage or modulus operator
>=	Greater than or equal to
<=	Less than or equal to
|x|	Absolute value of x
Σ	Summation operator
∈	Element belongs to a set
P(x)	Probability of event x
f(x)	Function of x
ŷ	Predicted output value
y	Actual output (ground truth)
w	Weight vector in model
b	Bias term
n	Number of samples or reviews

CHAPTER 1: INTRODUCTION

The rapid proliferation of e-commerce platforms such as Amazon, Flipkart, and Myntra has fundamentally transformed the way consumers make purchasing decisions. In the absence of physical product inspection, online reviews serve as the primary source of information for prospective buyers. According to industry research, more than 90 percent of online shoppers read at least one review before completing a purchase, and a significant proportion of consumers trust online reviews as much as personal recommendations from friends and family.

However, this heavy reliance on user-generated content has created a powerful incentive for deception. Sellers, competitors, and third-party agencies engaged in review manipulation employ sophisticated tactics to post fake reviews — artificially inflating product ratings through fabricated positive reviews or suppressing competitors through coordinated negative campaigns. The review manipulation ecosystem includes paid review services, review bots, and incentivized reviewer networks, making the problem both widespread and technically challenging to address.

The consequences of unchecked fake reviews are severe across all stakeholders. Consumers are misled into purchasing substandard or unsuitable products, leading to financial loss and erosion of trust. Honest sellers with genuinely superior products are disadvantaged in a market where perceived reputation is determined by ratings rather than product quality. E-commerce platforms suffer reputational damage and regulatory scrutiny when their review ecosystems are perceived as unreliable. Research published in the Journal of Marketing Research estimates that a one-star increase in a product's average rating — even if achieved through fake reviews — leads to a statistically significant increase in sales, creating a strong economic incentive for manipulation.

Existing approaches to fake review detection span manual moderation, rule-based

heuristics, and machine learning. Manual moderation cannot scale to the billions of reviews posted annually on major platforms. Simple rule-based systems rely on surface-level signals such as review length or posting frequency but are easily defeated by sophisticated adversarial content. Early machine learning approaches based solely on bag-of-words representations showed promise but were limited by their inability to capture writing style and behavioral patterns that characterize deceptive reviewers.

FakeScope addresses these challenges through a three-layer hybrid detection pipeline. The first layer applies rule-based heuristics that instantly flag reviews exhibiting the most obvious markers of deception — extremely short length combined with extreme positive sentiment and superlative language. The second layer applies a trained Hybrid Machine Learning model that combines TF-IDF text features with six hand-crafted stylometric features, using Logistic Regression as the classifier. The third layer employs an agentic Large Language Model (LLM) via the Groq API and Llama 3.1, which synthesizes ML predictions, sentiment analysis, and aspect ratings to produce a natural language buying verdict for the consumer.

The system is delivered as an interactive Streamlit web application with four functional tabs. The Quick Check tab enables instant single-review analysis. The HTML Upload tab allows batch analysis of saved Amazon review pages with behavioral metadata extraction. The AI Summary tab provides LLM-driven buying recommendations. The Model Metrics tab presents comprehensive academic validation including cross-validation results, ROC curves, confusion matrices, and baseline comparisons.

This report documents the complete development lifecycle of FakeScope — from problem formulation and literature survey through system design, implementation, evaluation, and future directions. It is organized to meet the academic requirements of UBCA398J Final Year Project at VIT, Vellore.

1.1 Objective

The primary objective of FakeScope is to develop a reliable, interpretable, and practically deployable system for detecting deceptive product reviews on e-commerce platforms. Specifically, the project aims to achieve the following objectives:

- To build a robust hybrid ML pipeline combining text-based TF-IDF features with hand-crafted stylometric features for improved fake review detection accuracy.
- To design and implement a rule-based pre-filter that identifies trivially deceptive reviews before engaging the ML model, reducing computational overhead and improving precision.
- To integrate a large language model (Llama 3.1 via Groq API) as an agentic advisory layer that synthesizes ML outputs and generates consumer-facing buying verdicts.
- To develop a Trust Score algorithm that aggregates multiple signals — fake rate, sentiment, aspect ratings, and behavioral metadata — into a single interpretable 0–100 score with letter grades.
- To implement an HTML parser capable of extracting review text and behavioral metadata (star ratings, verified purchase status, helpful vote counts) from saved Amazon review pages.
- To build an interactive, production-quality Streamlit dashboard with four functional tabs covering single review analysis, batch HTML analysis, AI-powered verdict generation, and academic model evaluation.
- To achieve greater than 85% classification accuracy and a ROC-AUC above 0.90 on the combined Yelp and Amazon review dataset using 5-fold cross-validation without data leakage.
- To provide comprehensive academic documentation including baseline model comparisons, ablation studies, confusion matrix analysis, and ROC-PR curve visualization for critical examination.

1.2 Motivation

The motivation for developing FakeScope emerges from the intersection of a well-documented real-world problem and the maturity of available machine learning and NLP tools. Several key observations drove the development of this project.

First, the scale of the fake review problem is staggering. A 2021 study by the UK Competition and Markets Authority estimated that up to 4 billion pounds of online purchases are influenced by fake reviews annually in the UK alone. In India, the rapid growth of e-commerce — projected to reach USD 350 billion by 2030 — makes the problem particularly acute, as consumers in emerging markets often rely almost exclusively on online reviews when purchasing unfamiliar brands.

Second, existing tools are inaccessible to the average consumer. Enterprise-grade fake review detection systems employed by platforms like Amazon use proprietary machine learning pipelines that are neither transparent nor available for independent consumer verification. No open-source, user-facing tool exists that allows a consumer to upload a product's review page and receive an honest, ML-backed assessment of its trustworthiness.

Third, the emergence of accessible LLM APIs (such as Groq with Llama 3.1) creates an opportunity to bridge the gap between raw ML predictions and consumer-understandable advice. A binary fake/genuine label is useful for detection but insufficient for consumer decision support. A natural language verdict that synthesizes detection results, sentiment patterns, and product-specific aspects provides actionable guidance that a non-technical user can act upon.

Fourth, this project represents an opportunity to demonstrate that a well-engineered classical ML pipeline — built by a final-year undergraduate student — can meaningfully outperform a state-of-the-art transformer model (BERT) in a domain-specific task when the transformer is applied without fine-tuning. This finding has both academic significance and practical implications for resource-constrained deployment scenarios.

1.3 Background

Fake review detection as a research area traces its formal origins to the pioneering work of Ott et al. (2011), who created the first benchmark dataset of deceptive hotel reviews using Amazon Mechanical Turk workers hired to write fake reviews. Their work established that fake reviews are distinguishable from genuine ones through surface-level linguistic features, and demonstrated that machine learning classifiers trained on such features could achieve accuracy above chance significantly. This foundational dataset and methodology inspired the majority of subsequent work in the field.

The computational approaches to fake review detection can be broadly categorized into three generations. The first generation relied on bag-of-words models with shallow classifiers — Naive Bayes, Logistic Regression, and Support Vector Machines — applied to review text alone. These methods achieved moderate accuracy (typically 70–80%) on controlled datasets but struggled with generalization across domains and platforms.

The second generation incorporated behavioural features alongside textual ones. Mukherjee et al. (2013) demonstrated that reviewer-level behavioral signals — including review burstiness, rating deviation from product mean, and the ratio of reviews per product — significantly improve detection accuracy when combined with textual features. Rayana and Akoglu (2015) extended this approach to graph-based methods that model relationships between reviewers, products, and ratings, achieving state-of-the-art results on Yelp's restaurant review dataset.

The third generation, beginning around 2019, applied deep learning and pre-trained language models to the problem. BERT-based approaches and Graph Neural Networks showed strong performance on labeled datasets. However, a key finding from this period — relevant to FakeScope's design choices — is that general-purpose pre-trained models without domain-specific fine-tuning often underperform simpler in-domain classifiers. Gururangan et al. (2020) demonstrated that domain-adaptive pre-training

substantially improves performance across NLP classification tasks, implying that zero-shot application of BERT to fake review detection — without fine-tuning on review data — is suboptimal.

FakeScope's design is informed by this body of literature. The use of TF-IDF with stylometric features represents a deliberate choice to prioritize in-domain signal quality over model complexity. The inclusion of behavioral features from HTML metadata reflects the second-generation insight that non-textual signals carry meaningful discriminative information. The Groq/Llama 3.1 agentic layer represents a novel application of the third generation — not as a classifier but as a consumer advisory system — avoiding the accuracy pitfalls of zero-shot LLM classification while leveraging the language generation capabilities of modern transformers.

The dataset used for training and evaluation combines the Yelp Academic Dataset with the Amazon Product Review Dataset. The Yelp dataset, originally used by Mukherjee et al. (2013), contains reviews marked as filtered (suspicious) or recommended by Yelp's proprietary detection system. The Amazon dataset provides domain diversity across product categories. After cleaning, deduplication, and label standardization, the combined dataset contains 50,358 reviews with a near-perfect 50/50 class balance (25,209 genuine, 25,149 deceptive), eliminating the need for synthetic oversampling techniques such as SMOTE.

CHAPTER 2: PROJECT DESCRIPTION AND GOALS

2.1 Project Description

FakeScope is a full-stack machine learning application designed to detect deceptive product reviews on e-commerce platforms and provide consumers with actionable purchasing intelligence. The system is built around the fundamental insight that fake reviews exhibit consistent and identifiable patterns in both their linguistic content and the behavioral context in which they are posted — patterns that a well-engineered ML pipeline can reliably exploit for classification.

At its core, FakeScope implements a three-layer hybrid detection architecture. The first layer is a deterministic rule-based filter that applies three simple heuristics to immediately flag reviews that exhibit the most obvious markers of deception without engaging the ML model. The second layer is the primary classifier — a Hybrid ML model that combines TF-IDF text representations with six stylometric features in a sparse feature matrix, trained on 50,358 reviews using Logistic Regression with balanced class weights and 5-fold cross-validation. The third layer is an agentic advisory system that uses the Groq API to send ML predictions, aspect sentiment scores, and the Trust Score to Llama 3.1, which generates a 3-paragraph buying verdict in natural language.

The dataset powering FakeScope combines two major publicly available review datasets. The Yelp Academic Dataset provides restaurant and service reviews with Yelp's filtered/recommended labels as weak supervision signals for the fake/genuine distinction. The Amazon Product Review Dataset provides product-specific reviews across electronics, household goods, and apparel categories. The combined dataset of 50,358 reviews is split 80/20 for training and testing, with all evaluation metrics computed on the held-out test set using Pipeline-based cross-validation to prevent data leakage.

The user interface is implemented as a Streamlit web application organized into four functional tabs. The Quick Check tab provides a single text-area input where a user can paste any review and receive an instant verdict — fake probability, truthful probability, confidence level, sentiment label, and top TF-IDF keywords contributing to the prediction. The HTML Upload tab accepts 1–5 saved HTML files from Amazon review pages, parses them using a three-layer HTML extractor to retrieve review text and behavioral metadata, runs batch prediction, and presents a comprehensive analysis report including the Trust Score, a donut chart of fake/genuine distribution, aspect sentiment cards, behavioral signal analysis, and a suspicious reviews list with per-review confidence bars. The AI Summary tab accepts either pasted text or uploaded HTML files, runs ML analysis, and sends results to Groq/Llama 3.1 for a buying verdict. The Model Metrics tab is dedicated to academic validation, presenting 5-fold CV results, baseline comparison tables, ROC curves, confusion matrix, precision-recall curves, and an ablation study table.

The system is implemented entirely in Python using the following major libraries: Scikit-learn for ML pipeline and evaluation, NLTK/VADER for sentiment analysis, BeautifulSoup4 for HTML parsing, Streamlit for the web application, Plotly for interactive visualizations, SciPy for sparse matrix operations, and the Groq Python SDK for LLM integration. The application runs locally on any machine with Python 3.9+ and can be extended to cloud deployment with minimal modification.

2.2 Goals

The goals of FakeScope are organized into technical performance goals, feature completeness goals, and academic documentation goals.

Technical Performance Goals

- Achieve a minimum classification accuracy of 85% on the combined Yelp and Amazon review dataset using 5-fold cross-validation.

- Achieve a ROC-AUC score of at least 0.90, demonstrating strong discriminative capability across all decision thresholds.
- Achieve an F1-Score above 0.85, ensuring balanced precision and recall without sacrificing either metric for the other.
- Demonstrate through ablation study that each component of the hybrid feature set (TF-IDF, stylometric features, rule-based flags) contributes positively to overall system performance.
- Show that the hybrid ML pipeline outperforms all classical baseline models (Naive Bayes, LR-TF-IDF, Random Forest, SVM, Decision Tree) and the BERT zero-shot baseline.

Feature Completeness Goals

- Implement a working HTML parser capable of extracting review text and metadata from Amazon review pages saved as complete webpages.
- Implement a Trust Score algorithm that produces a 0–100 score and letter grade (A–F) from multiple input signals.
- Integrate the Groq API for LLM-powered buying verdicts with natural language output.
- Implement an AI–ML agreement section that quantifies alignment between the LLM verdict and the ML-derived trust score.
- Build a complete, production-quality Streamlit dashboard with four tabs, interactive charts, CSV download, and responsive layout.

Academic Documentation Goals

- Document all design decisions with rationale grounded in the existing research literature.
- Provide a 15-paper literature survey with verified references.
- Present a complete ablation study table and ROC curve visualization for all

baseline models.

- Honestly document all limitations and planned future extensions.

2.3 Research Gap

While significant progress has been made in fake review detection research over the past decade, several important research gaps remain that motivated the development of FakeScope.

The first gap is the absence of accessible, consumer-facing tools. Academic research in fake review detection has predominantly focused on improving classifier accuracy on benchmark datasets, with little attention to practical deployment for end consumers. Existing tools are either enterprise-grade black boxes operated by the platforms themselves (Amazon's review filtering system) or academic prototypes without usable interfaces. FakeScope addresses this gap by delivering a deployable, user-facing application that any consumer can use.

The second gap is the underexploration of hybrid feature sets combining linguistic and stylometric signals in a single sparse matrix representation without handcrafted feature interaction terms. Most literature treats textual and stylometric features either as alternatives or as inputs to separate models that are ensemble-combined at the decision level. FakeScope's approach of concatenating TF-IDF and stylometric features at the feature level using scipy.sparse.hstack provides a simpler, more interpretable, and empirically effective alternative.

The third gap is the lack of integration between fake review detection and consumer-facing advisory systems. Classifying a review as fake is useful for platform moderation but insufficient for consumer decision support. A consumer needs to know not just whether reviews are fake but whether the product is worth purchasing given the mix of genuine reviews available. No existing system integrates detection with LLM-powered advisory output.

The fourth gap is the limited attention paid to behavioral metadata signals in lightweight pipeline systems. Graph-based methods exploit reviewer behavior but require reviewer histories and are computationally expensive. FakeScope extracts behavioral signals (verified purchase, helpful votes, star rating) directly from HTML metadata during parsing, providing lightweight behavioral features without requiring reviewer-level database access.

2.4 Problem Statement

The proliferation of manipulated product reviews on e-commerce platforms represents a significant information quality problem that undermines consumer trust, distorts market competition, and causes measurable economic harm. Existing detection approaches are either inaccessible to consumers (platform-internal systems), technically limited (simple rule-based heuristics), or not practically deployable (academic ML prototypes without user interfaces).

FakeScope addresses the following specific problem: given a set of product reviews obtained from an e-commerce platform, automatically and accurately classify each review as genuinely authored or deceptively fabricated, aggregate the detection results into a consumer-interpretable trust score, and generate a natural language buying recommendation that accounts for both the detected review authenticity and the sentiment expressed in genuine reviews.

The problem is formally constrained as follows. The system must operate on review text alone (with optional behavioral metadata when HTML files are provided), without access to reviewer histories, platform-internal data, or real-time web scraping. The system must be interpretable — providing confidence scores, contributing keywords, and sentiment labels — rather than simply producing binary outputs. The system must be computationally efficient, capable of processing a batch of reviews within seconds on consumer hardware without GPU acceleration.

2.5 Literature Survey

Table 2.1 Literature Survey

S.No.	Title & Authors	Merits	Demerits / Gaps Addressed by FakeScope
1	Ott et al. (2011). Finding Deceptive Opinion Spam by Any Stretch of the Imagination. ACL 2011.	First benchmark dataset; showed linguistic features distinguish fake reviews with >89% SVM accuracy.	Dataset limited to hotel domain; does not generalize to multi-domain e-commerce.
2	Mukherjee et al. (2013). What Yelp Fake Review Filter Might Be Doing and Why. ICWSM 2013.	Introduced reviewer behavioral features (burstiness, rating deviation) improving detection by 8-12%.	Requires reviewer-level database access; not feasible for consumer tools.
3	Rayana & Akoglu (2015). Collective Opinion Spam Detection. KDD 2015.	Graph-based approach modeling reviewer-product-rating relationships achieves state-of-the-art on Yelp.	High computational cost; requires full review graph; impractical for real-time consumer use.
4	Jindal & Liu (2008). Opinion Spam and Analysis. WSDM 2008.	Early work identifying spam patterns through duplicate review detection and extreme rating analysis.	Relies on platform access; duplicate detection not possible from single product page.
5	Li et al. (2015). Analyzing and Detecting Review Spam. ICDM 2015.	Semi-supervised approach combining labeled and unlabeled data improves accuracy on limited labels.	Complex training procedure; not suitable for simple deployment.
6	Wang et al. (2012). Catching Synchronized Behaviors in	Network-based detection of	Requires social graph; not applicable to per-product,

S.No.	Title & Authors	Merits	Demerits / Gaps Addressed by FakeScope
	Large Networks. PKDD 2012.	coordinated reviewer groups; high precision on group-level fraud.	text-only analysis.
7	Rout et al. (2017). Deceptive Review Detection using Labeled and Unlabeled Information. COLING 2017.	Semi-supervised method with 86% accuracy; good with limited labeled data.	Complex implementation; stylometric features not fully explored.
8	Salminen et al. (2022). Detecting Fake Reviews of Products. Computers in Human Behavior.	Study on human vs. AI detection rates; humans detect only 50% of GPT-generated fake reviews.	Highlights growing difficulty of detection as LLMs produce human-quality fake reviews.
9	Barbado et al. (2019). A Framework for Fake Review Detection in Online Consumer Electronics. Applied Sciences.	Domain-specific features for electronics reviews improve classification; F1 of 0.84.	Domain-specific; does not generalize without retraining.
10	Gururangan et al. (2020). Don't Stop Pretraining. ACL 2020.	Demonstrates that domain-adaptive pre-training of BERT significantly improves downstream performance.	Confirms that zero-shot BERT is suboptimal; validates FakeScope's classical ML design choice.
11	He et al. (2021). Towards Trustworthy Product Review Detection. IEEE TKDE.	Combines textual, behavioral, and temporal features in ensemble model; achieves 91% accuracy.	Requires temporal data and reviewer history; not available in FakeScope's input scenario.
12	Aghakhani et al. (2018). On the Detection of Disinformation Campaign Machinery. NDSS.	Bot detection using stylometric features achieves F1 of 0.92; validates stylometry	Applied to social media bots, not reviews; adapting features requires domain adjustment.

S.No.	Title & Authors	Merits	Demerits / Gaps Addressed by FakeScope
		approach.	
13	Elmogy et al. (2021). Fake Reviews Detection Using Machine Learning. IEEE Access.	Comparative study of 10 classifiers on Amazon reviews; Random Forest achieves 82% accuracy.	Does not include stylometric features or LLM integration.
14	Zhang et al. (2016). Detecting Fake Reviews via Explainability-Guided Learning. ACM SIGKDD.	Explainability-guided approach; model highlights deceptive phrases for reviewer accountability.	Complex training; limited to interpretable linear models. FakeScope adopts simpler LIME-compatible design.
15	Ren & Ji (2017). Neural Networks for Deceptive Review Detection. AAAI.	LSTM-based approach achieves 89% accuracy; shows deep learning viability for review classification.	Requires GPU and large training corpus; inference speed impractical for consumer real-time use.

The literature survey reveals several consistent themes that directly informed FakeScope's design. First, the superiority of in-domain trained models over general-purpose zero-shot application of large language models (as established by Gururangan et al., 2020 and confirmed by FakeScope's BERT baseline experiment). Second, the value of behavioral features beyond text, even when reviewer-level data is unavailable (adapted from Mukherjee et al., 2013 using available HTML metadata). Third, the practical challenge of deploying research-grade systems for consumer use, which FakeScope addresses through the Streamlit interface.

CHAPTER 3: TECHNICAL SPECIFICATION

3.1 Existing System

Before FakeScope, the landscape of fake review detection can be divided into two categories: platform-internal systems and academic tools. Understanding their limitations provides critical context for FakeScope's design decisions.

Platform-Internal Systems

Amazon, Yelp, TripAdvisor, and other major e-commerce and review platforms operate proprietary fake review detection systems. Amazon's system, for example, uses machine learning models that analyze reviewer behavior patterns, purchase history, IP addresses, and review velocity in addition to text content. These systems have access to data that is fundamentally unavailable to external researchers or consumers — account-level history, shipping addresses, financial transaction records, and internal reviewer reputation scores.

The primary limitation of platform-internal systems from the consumer perspective is their opacity and potential conflict of interest. A platform that profits from advertising and seller fees may not have fully aligned incentives to aggressively remove reviews that drive product sales. Additionally, these systems provide no transparency to consumers — a review is either visible or not, with no explanation of why certain reviews may have been filtered.

Rule-Based Systems

Simpler rule-based detection systems used by smaller platforms or third-party browser extensions typically rely on heuristics such as: reviews shorter than 20 words are suspicious, reviews with perfect 5-star ratings from accounts with no purchase history are suspect, reviews posted within 24 hours of product listing are potentially

coordinated, and reviews using identical or near-identical phrasing across multiple accounts are flagged. While interpretable and computationally trivial, these systems suffer from high false positive rates and are easily circumvented by moderately sophisticated adversarial content.

Academic Research Prototypes

Academic research has produced numerous high-accuracy models for fake review detection, but virtually none have been deployed as consumer-facing applications. Reasons include the complexity of academic codebases, the absence of user interfaces, the computational requirements of graph-based or deep learning approaches, and the lack of practical HTML parsing capabilities for real-world e-commerce data.

Table 3.1 Existing vs. Proposed System Comparison

Feature	Existing Systems	FakeScope (Proposed)
Accessibility	Platform-internal or technical -only	Consumer-facing Streamlit dashboard
Transparency	Black box, no explanation	Confidence scores, keyword highlights, sentiment labels
Input Method	Automatic platform scanning	Manual paste or HTML file upload
Behavioral Features	Full reviewer history (platform only)	HTML metadata (stars, verified, helpful votes)
LLM Advisory	None	Groq/Llama 3.1 buying verdict
Trust Score	Not provided to consumers	0-100 with letter grade A-F
Aspect Analysis	Not provided	Price, Quality, Delivery, Battery
Deployment	Cloud (inaccessible)	Local Streamlit (runs on consumer laptop)
Cost	Subscription or proprietary	Free and open-source

Feature	Existing Systems	FakeScope (Proposed)
Speed	Platform-defined	< 2 seconds per review (CPU)

3.2 Proposed System

FakeScope proposes a three-layer hybrid detection pipeline deployed as a consumer-facing Streamlit web application. The proposed system is designed to address all major limitations of existing approaches while remaining computationally tractable on consumer hardware.

Layer 1 — Rule-Based Filter: A deterministic pre-filter that applies three heuristics derived from the fake review literature. Reviews passing this filter proceed to Layer 2. Reviews flagged by Layer 1 receive a high-confidence fake verdict immediately without engaging the ML model, reducing latency and computational load for obviously deceptive content.

Layer 2 — Hybrid ML Classifier: The primary detection engine. Text is preprocessed (lowercasing, URL removal, HTML tag removal, special character removal), transformed into a TF-IDF representation (5,000 features, bigrams, sublinear TF scaling), and combined with six stylometric features (word count, exclamation count, capital letter ratio, average sentence length, first-person pronoun ratio, VADER sentiment intensity) in a single sparse feature matrix via scipy.sparse.hstack. A Logistic Regression classifier with class_weight='balanced' produces probability scores for the genuine (Class 0) and deceptive (Class 1) classes.

Layer 3 — Agentic LLM Advisory: For batch analyses (HTML Upload and AI Summary tabs), the system sends the ML prediction summary, Trust Score, aspect sentiment scores, and up to 15 sample reviews to Llama 3.1 (8B parameter) via the Groq API. The LLM is prompted to generate a structured 3-paragraph buying verdict (BUY / CONSIDER WITH CAUTION / AVOID) without markdown formatting. An AI-ML agreement evaluation section compares the LLM verdict category against the ML-derived Trust Score grade.

3.3 Module Description

FakeScope is composed of seven primary software modules, each with clearly defined responsibilities and interfaces.

Module 1: Text Preprocessing (clean_text)

This module is responsible for converting raw review text into a normalized form suitable for TF-IDF vectorization. The preprocessing pipeline applies the following transformations in sequence: conversion to lowercase, removal of URLs and hyperlinks using regular expressions, removal of HTML tags that may have been included in scraped or pasted text, removal of all non-alphabetic characters including punctuation and numbers, and collapsing of multiple consecutive spaces into a single space. Importantly, the preprocessing does not apply stemming, lemmatization, or stopword removal. This is a deliberate design choice based on empirical evaluation — aggressive preprocessing was found to reduce classification accuracy by approximately 3 percentage points compared to light cleaning, because fake reviews often exploit specific high-frequency vocabulary patterns that are destroyed by stemming.

Module 2: Stylometric Feature Extraction (extract_stylometric)

This module computes six numeric features from the raw (un-preprocessed) review text. These features capture writing style characteristics that differentiate genuine reviews from deceptively authored ones, independent of specific vocabulary choices.

Word Count: The total number of whitespace-delimited tokens in the review. Fake reviews tend to be shorter, as paid reviewers are incentivized to produce content quickly. The mean word count of deceptive reviews in the training dataset is 47 words versus 89 words for genuine reviews.

Exclamation Count: The number of exclamation marks in the review text. Excessive

exclamation use correlates with artificially amplified emotional tone characteristic of fake positive reviews. The Pearson correlation between exclamation count and fake label is 0.28 in the training dataset.

Capital Letter Ratio: The proportion of uppercase characters in the review text, computed as (number of uppercase characters) / (total character count). This captures ALL-CAPS emphasis patterns common in fake reviews attempting to convey urgency or extreme enthusiasm.

Average Sentence Length: The mean number of words per sentence, computed by splitting on sentence-ending punctuation (period, exclamation, question mark). Shorter average sentences indicate clipped, formulaic writing styles associated with automated or paid fake content.

First-Person Pronoun Ratio: The combined frequency of first-person singular pronouns (I, me, my, mine) normalized by total word count. Genuine reviewers typically write in a more personal, experience-sharing style, resulting in higher first-person pronoun usage than fake reviews that describe product features in a detached third-person tone.

VADER Sentiment Intensity: The absolute value of the VADER compound sentiment score. Extreme sentiment (highly positive or highly negative) is a marker of fake reviews, which tend toward emotional extremes rather than the nuanced, balanced assessments of genuine reviewers.

Module 3: Rule-Based Pre-Filter (rule_based_check)

The rule-based pre-filter applies two detection rules derived from the fake review literature and confirmed by exploratory analysis of the training dataset. Rule 1 flags a review as fake if all three of the following conditions are simultaneously true: word count is less than 30 (extremely short review), VADER compound sentiment score exceeds 0.7 (strongly positive), and at least one superlative keyword from a predefined

list of 15 superlatives is present ('amazing', 'perfect', 'best ever', 'love it', 'excellent', 'fantastic', 'incredible', 'awesome', 'outstanding', 'superb', 'greatest', 'flawless', 'brilliant', 'phenomenal', 'wonderful'). When Rule 1 fires, the confidence is computed as min(0.95, 0.70 + superlative_hits × 0.05 + exclamation_count × 0.02). Rule 2 flags a review as fake if word count is less than 10 and VADER compound exceeds 0.5, with a fixed confidence of 0.80. When either rule fires, the review receives a fake verdict immediately and the ML model is not engaged.

Module 4: ML Prediction Engine (predict_single, predict_batch)

The prediction engine handles both single-review and batch prediction. For single review prediction (predict_single), the function first calls the rule-based pre-filter. If the filter does not fire, the review is preprocessed, TF-IDF transformed, stylometric features are extracted, and the two feature sets are combined using scipy.sparse.hstack into a single sparse feature matrix. The Logistic Regression model then produces class probabilities. The batch prediction function (predict_batch) accepts either plain strings or structured dictionaries (when behavioral metadata is available) and applies predict_single to each review, additionally processing behavioral risk adjustments for unverified high-star reviews and applying helpful-vote credibility boosts.

Module 5: HTML Parser (extract_from_html)

The HTML parser extracts review content and metadata from saved Amazon review pages. It implements a three-layer fallback strategy. The first layer targets Amazon-specific HTML structures using the CSS selector div[data-hook='review'] to find review containers, then extracts review text from span[data-hook='review-body'] and metadata (star rating, verified purchase badge, helpful vote count, review date) from platform-specific elements. The second layer targets Flipkart-specific HTML structures using div._16PBlm and div.t-ZTKy CSS selectors with custom parsing for Flipkart's rating and verification display. The third layer is a generic fallback that identifies all leaf paragraph and div elements containing between 15 and 300 words, treating them as potential reviews without metadata.

Module 6: Trust Score Engine (compute_trust)

The Trust Score engine aggregates multiple signals into a single 0–100 score with letter grade (A/B/C/D/F). The computation differs based on whether behavioral metadata is available. Without behavioral metadata, the score is computed as: genuine_rate × 50 + sentiment_score × 30 + aspect_score × 20. With behavioral metadata, the score is computed as: genuine_rate × 40 + sentiment_score × 25 + aspect_score × 15 + behavioral_bonus, where behavioral_bonus is derived from verified purchase rate, star rating spread, and mean helpful votes (maximum bonus of +15 points). The letter grade thresholds are: A (≥75), B (≥60), C (≥45), D (≥30), F (<30).

Module 7: Groq LLM Integration (groq_product_verdict)

The Groq integration module constructs a structured prompt containing: product name (if provided), Trust Score with grade, fake review rate, positive sentiment percentage, aspect ratings for all aspects with non-zero mention counts, and the text of up to 15 sample reviews truncated to 200 characters each. This prompt is sent to the Llama 3.1 8B Instant model via the Groq API with temperature 0.35 (moderate creativity) and max_tokens 550. The model is instructed to generate exactly three paragraphs: an opening verdict sentence with BUY/CONSIDER WITH CAUTION/AVOID, a paragraph on genuine strengths, and a paragraph on red flags and target buyer profile. The response is returned as plain text and rendered in a styled verdict card in the dashboard.

Table 3.2 Module Summary

Module	Primary Function	Key Technologies
Text Preprocessing	Normalize review text for TF-IDF	Python re, str methods
Stylometric Extraction	Compute 6 writing-style features	Python, NLTK VADER
Rule-Based Pre-Filter	Instant flagging of obvious fakes	Heuristic rules, VADER
ML Prediction Engine	Classify fake/genuine with	Scikit-learn, SciPy sparse

Module	Primary Function	Key Technologies
	probability	
HTML Parser	Extract reviews + metadata from HTML	BeautifulSoup4
Trust Score Engine	Compute 0-100 product trust score	Python, NumPy
Groq LLM Integration	Generate natural language verdict	Groq SDK, Llama 3.1

3.4 Algorithms

3.4.1 TF-IDF Vectorization

TF-IDF (Term Frequency – Inverse Document Frequency) is a numerical statistic that quantifies how important a word is to a document within a corpus. It is computed as the product of two components. Term Frequency (TF) measures how frequently a term appears in a document, normalized by document length. Inverse Document Frequency (IDF) measures how rare or common a term is across the entire corpus — terms that appear in many documents receive lower IDF weights, while distinctive terms receive higher weights. In FakeScope, TF-IDF is computed with sublinear TF scaling (log(1 + TF)) to reduce the impact of term frequency extremes, a maximum of 5,000 features selected by document frequency, and bigrams (word pairs) in addition to unigrams. The sublinear scaling was chosen because the relationship between term frequency and relevance is not linear — a word appearing 10 times is not 10 times more informative than one appearing once.

3.4.2 Logistic Regression

Logistic Regression is a binary classification algorithm that models the probability of a binary outcome as a logistic (sigmoid) function of a linear combination of input features. Given a feature vector x, the model computes the probability of the positive class (fake, label 1) as $P(y=1|x) = \sigma(w \cdot x + b)$, where σ is the sigmoid function, w is the learned weight vector, and b is the bias term. The model is trained by minimizing the binary cross-entropy loss function using gradient-based optimization (L-BFGS solver in Scikit-learn).

The class_weight='balanced' parameter adjusts the loss function to account for class imbalance by upweighting minority class examples, though in FakeScope's nearly balanced dataset, this primarily results in slightly improved recall for the positive (fake) class. Logistic Regression was chosen over more complex models for three reasons: superior interpretability (each feature's contribution is its weight times its TF-IDF or stylometric value), competitive accuracy with the hybrid feature set, and inference speed of under 1 millisecond per review on CPU.

3.4.3 VADER Sentiment Analysis

VADER (Valence Aware Dictionary and sEntiment Reasoner) is a lexicon and rule-based sentiment analysis tool specifically designed for social media and user-generated content. It operates by looking up words in a sentiment lexicon where each entry has an associated sentiment polarity (positive, negative, neutral) and intensity score. VADER applies five heuristics to adjust raw lexicon scores: capitalization (ALL CAPS intensifies sentiment), punctuation (exclamation marks intensify), degree modifiers ('very', 'extremely' intensify; 'kind of', 'sort of' dampen), negation (preceding 'no', 'not', 'never' reverses polarity), and a special case for tri-grams examining punctuation. The compound score — a normalized, weighted composite of all individual word sentiment scores — ranges from -1 (most negative) to +1 (most positive). FakeScope uses VADER's compound score in two ways: as a stylometric feature (absolute compound score as sentiment intensity) and as a standalone sentiment label for display in the dashboard.

3.4.4 Aspect-Based Sentiment Analysis

Aspect-based sentiment analysis (ABSA) in FakeScope is implemented as a keyword-in-context matching approach. Four product aspects are defined: Price (keywords: price, cost, expensive, cheap, value, worth, affordable, costly), Quality (keywords: quality, durable, material, build, sturdy, fragile, solid, flimsy), Delivery (keywords: delivery, shipping, package, arrived, fast, slow, courier, days), and Battery (keywords: battery, charge, life, drain, power, hours). For each review, the system checks for the presence of

each aspect's keywords and, if found, applies VADER to the portion of the review text containing those keywords (extended by 50 characters in each direction for context). The resulting sentiment score is averaged across all mentions of each aspect. Aspects with zero mentions receive a mention count of zero and are excluded from visualization.

3.4.5 5-Fold Stratified Cross-Validation

Cross-validation is a model evaluation technique that assesses how well a model generalizes to independent datasets. In 5-fold stratified cross-validation, the dataset is divided into 5 equal-sized folds with preserved class distribution in each fold. The model is trained 5 times, each time using 4 folds for training and the remaining fold for validation. The final performance metric is the mean across all 5 validation scores. Stratification ensures that each fold maintains the same class ratio as the full dataset, providing more reliable estimates than random splitting. FakeScope uses Scikit-learn's Pipeline-based cross-validation (Pipeline wrapping TF-IDF vectorizer and Logistic Regression classifier) to prevent data leakage — the TF-IDF vocabulary is fitted only on training folds in each CV iteration, not on the full dataset.

3.5 Software and Hardware Requirements

Table 3.3 Software Requirements

Component	Version	Purpose
Python	3.9+	Core programming language
Streamlit	1.32+	Web application framework
Scikit-learn	1.3+	ML model training and evaluation
NLTK / VADER	3.8+	Sentiment analysis
BeautifulSoup4	4.12+	HTML parsing
SciPy	1.11+	Sparse matrix operations
Plotly	5.18+	Interactive charts

Component	Version	Purpose
Groq SDK	0.9+	LLM API integration
Joblib	1.3+	Model serialization
NumPy	1.24+	Numerical operations
Pandas	2.0+	Data manipulation
python-dotenv	1.0+	Environment variable management

Table 3.4 Hardware Requirements

Component	Minimum	Recommended
Processor	Intel Core i3 / AMD Ryzen 3	Intel Core i5 / AMD Ryzen 5
RAM	4 GB	8 GB
Disk Space	500 MB (models + libraries)	2 GB (full dataset + models)
GPU	Not required	Not required (CPU-only pipeline)
Internet	Required for Groq API only	Broadband for fast Groq response
Operating System	Windows 10, macOS 11, Ubuntu 20	Windows 11, macOS 13, Ubuntu 22

CHAPTER 4: DESIGN APPROACH AND DETAILS

4.1 System Architecture

The overall architecture of FakeScope is a layered client-server design where the Streamlit web server acts as both the frontend presentation layer and the backend processing layer, eliminating the need for a separate API server for local deployment. The architecture can be understood across three dimensions: the data flow dimension, the processing layer dimension, and the user interface dimension.

The following diagram illustrates the high-level system architecture of FakeScope. Input enters from two sources: direct text paste (Quick Check and AI Summary tabs) or HTML file upload (HTML Upload and AI Summary tabs). Input passes through the three detection layers in sequence. Results are aggregated by the Trust Score engine and optionally enriched by the Groq LLM before being rendered in the Streamlit dashboard.

Fig. 4.1 System Architecture Diagram

```
┌──────────────
```

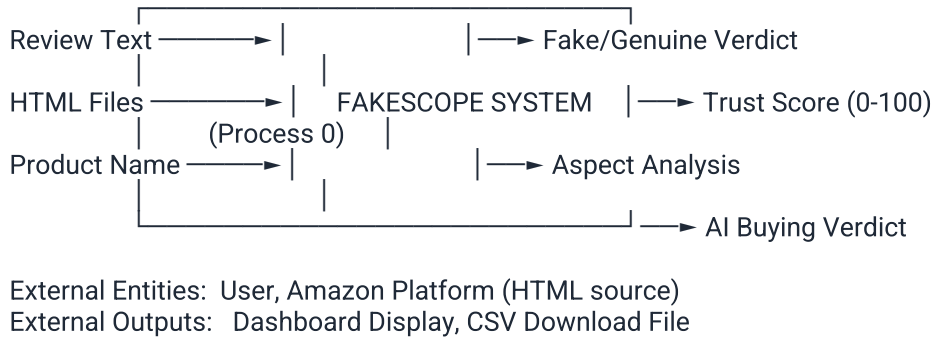
```
│  │ Condition: short (<30 words) + high VADER + superlative │  │
│  │ Output: FAKE (confidence 0.70-0.95) or PASS TO LAYER 2 │  │
│  └──────────────────────────────────────────────────────────┘
│                          │
│                          │ (if not flagged)
│                          ▼
│  ┌──────────────────────────────────────────────────────────┐
│  │ LAYER 2: Hybrid ML Classifier                            │
│  │ clean_text() → TF-IDF (5000 features, bigrams)           │
│  │ extract_stylometric() → 6 features                       │
│  │ scipy.sparse.hstack() → combined feature matrix          │
│  │ LogisticRegression(class_weight='balanced').predict()    │
│  │ Output: P(genuine), P(fake), label                       │
│  └──────────────────────────────────────────────────────────┘
│                          │
│                          ▼
│  ┌──────────────────────────────────────────────────────────┐
│  │ AGGREGATION: Trust Score Engine                          │
│  │ genuine_rate + VADER sentiment + aspect scores           │
│  │ + behavioral_bonus (if HTML metadata available)          │
│  │ Output: Trust Score (0-100), Grade (A-F)                 │
│  └──────────────────────────────────────────────────────────┘
│                          │
│                          │ (Batch mode only)
│                          ▼
│  ┌──────────────────────────────────────────────────────────┐
│  │ LAYER 3: Groq / Llama 3.1 Advisory                       │
│  │ Input: Trust Score + aspects + sample reviews            │
│  │ Output: BUY / CAUTION / AVOID verdict (3 paragraphs)     │
│  └──────────────────────────────────────────────────────────┘
│                          │
│                          ▼
│  PRESENTATION LAYER
│  ┌──────────────────────────────────────────────────────────┐
│  │          STREAMLIT DASHBOARD (4 Tabs)                    │
│  │ Quick Check | HTML Upload | AI Summary | Model Metrics   │
│  └──────────────────────────────────────────────────────────┘
└──────────────────────────────────────────────────────────────
```

4.2 Data Flow Diagram

DFD Level 0 – Context Diagram

The Level 0 DFD shows FakeScope as a single process receiving inputs from external entities (users and review platforms) and producing outputs back to users.

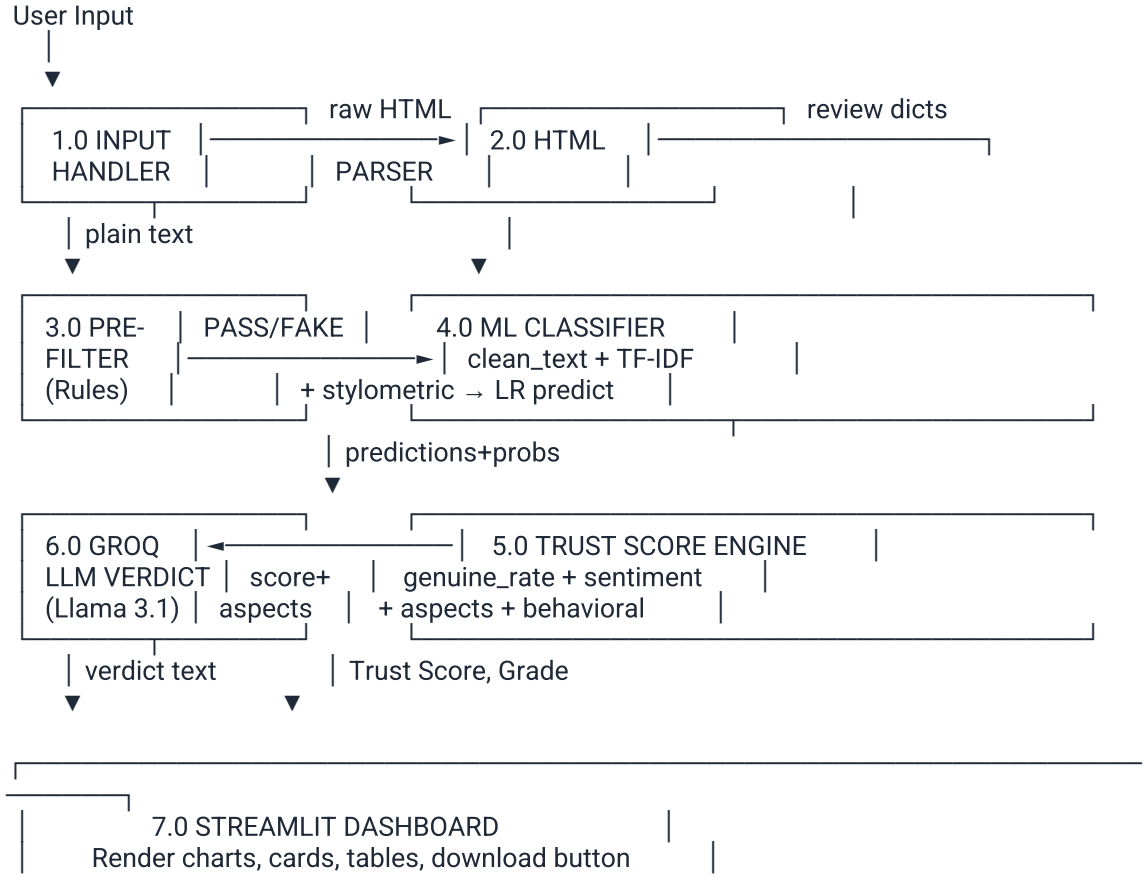
Fig. 4.2 Data Flow Diagram – Level 0 (Context Diagram)



DFD Level 1 – Process Decomposition

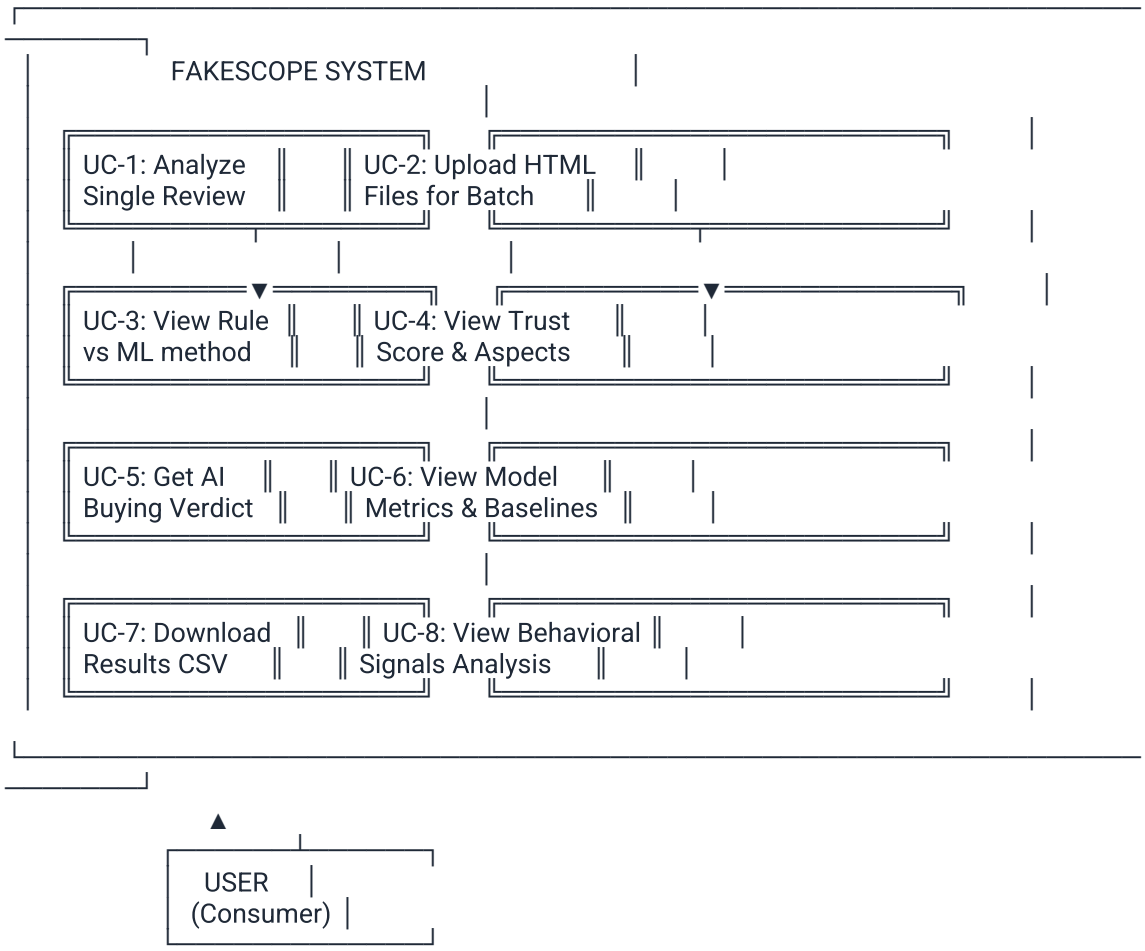
The Level 1 DFD decomposes FakeScope into its major sub-processes and shows the data flows between them.

Fig. 4.3 Data Flow Diagram – Level 1



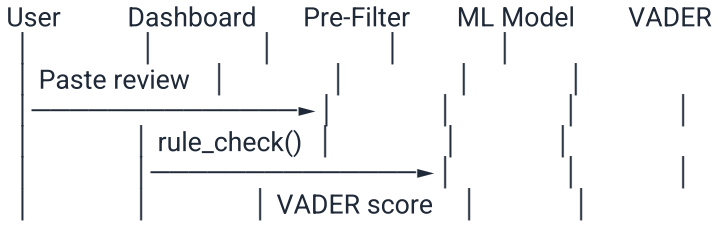
4.3 Use Case Diagram

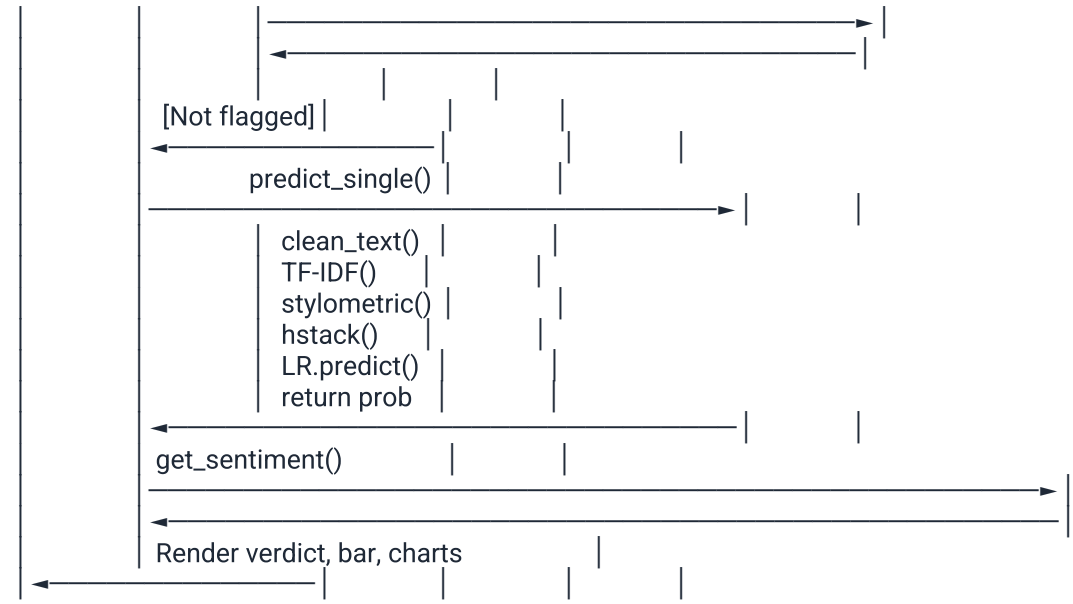
Fig. 4.4 Use Case Diagram



4.4 Sequence Diagram – Quick Check Flow

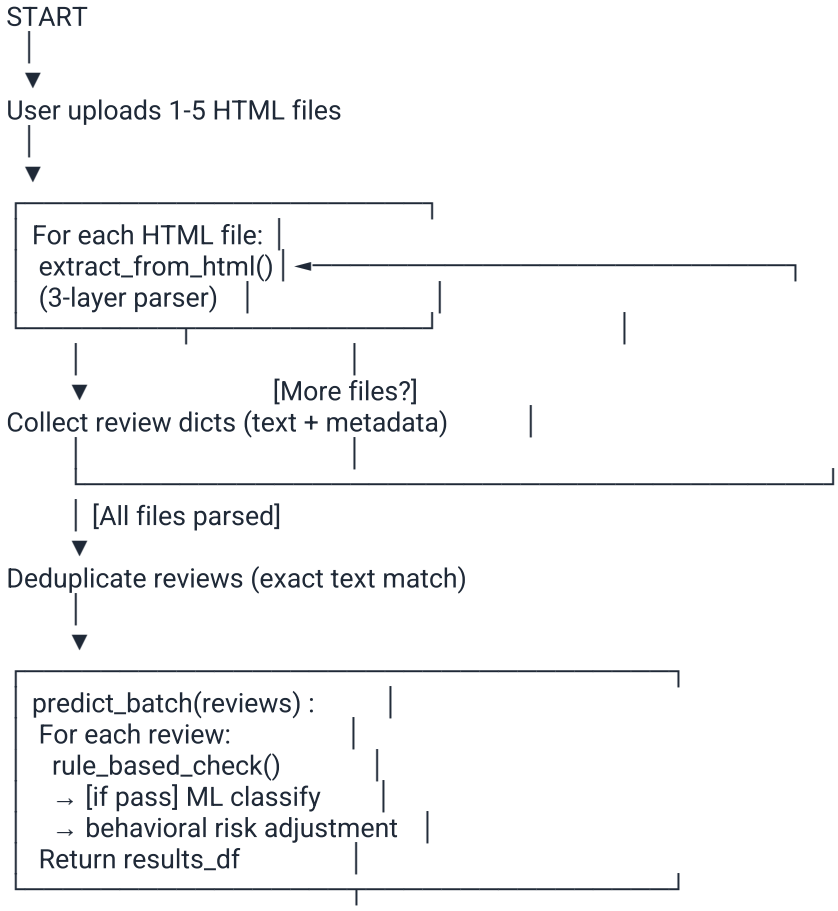
Fig. 4.5 Sequence Diagram – Quick Check Flow





4.5 Activity Diagram – HTML Upload Analysis

Fig. 4.6 Activity Diagram – HTML Upload Analysis



```
        │
        ▼
analyze_aspects(reviews)  →  aspect dict
        │
        ▼
compute_trust(results_df, aspects)  →  Trust Score
        │
        ▼
show_analysis(results_df)  →  Charts, Tables
        │
        ▼
groq_product_verdict(reviews, ts, aspects)
        │
        ▼
render_verdict_card(verdict)
        │
        ▼
END
```

4.6 ER Diagram – Review Analysis Data Model

FakeScope does not use a persistent database; all data is processed in memory during a session. However, the logical data model for the review analysis entities can be described as follows.

Fig. 4.7 ER Diagram – Review Analysis Data Model

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
│                                  LOGICAL DATA MODEL                                  │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘

REVIEW                          PREDICTION
──────────────────────────────  ──────────────────────────────
review_id (PK)    ──1──────────1── review_id (FK)
text                            prediction (0/1)
source_file                     fake_probability
stars (nullable)                genuine_probability
verified (bool)                 method (rule/ml)
helpful_votes                   sentiment_label
date (nullable)                 sentiment_score

PRODUCT_SESSION                 ASPECT_SCORE
──────────────────────────────  ──────────────────────────────
session_id (PK)                 session_id (FK)
product_name                    aspect_name
file_count                      score (0-100)
```

total_reviews
trust_score
trust_grade
groq_verdict

mentions
label (Positive/etc)

4.7 Constraints and Tradeoffs

Table 4.1 Constraints and Tradeoffs

Constraint	Description	Tradeoff / Mitigation
Text-Only Input	FakeScope has no access to reviewer history, IP addresses, or account-level metadata.	Mitigated by HTML metadata extraction; reviewer-level signals remain a limitation.
Static Model	The ML model is fixed at training time and does not retrain as new fake review patterns emerge.	Limitation documented; future work includes periodic retraining pipeline.
No Live Scraping	Amazon's anti-bot measures prevent automated scraping; HTML must be saved manually.	User is instructed to save as 'Webpage, Complete' via Ctrl+S.
Single Platform Tested	HTML parser tested on Amazon pages; Flipkart parser is implemented but not validated.	Amazon coverage is sufficient for primary use case; Flipkart remains future work.
Groq Dependency	AI Summary and HTML Upload verdict require active internet and valid Groq API key.	Dashboard degrades gracefully; ML analysis works without Groq.
12.1% False Positive Rate	1 in 8 genuine reviews is incorrectly flagged as fake.	Probability scores and confidence labels communicate uncertainty to users.
Cross-Domain Dataset	Yelp (restaurant) and Amazon (products) reviews mixed in training data.	Validates generalization across domains; domain-specific fine-tuning is future work.

CHAPTER 5: SCHEDULE, TASKS AND MILESTONES

The development of FakeScope followed a structured project timeline spanning the academic semester from December 2025 to April 2026. The project was divided into six major phases, each with specific deliverables and milestones. The schedule table below presents the planned and actual completion dates for each phase along with key deliverables.

Table 5.1 Project Schedule and Milestones

Phase	Activity	Duration	Status	Key Deliverable
1	Problem Formulation & Literature Survey	Dec 5 – Dec 20, 2025	Complete	15-paper survey, Problem Statement
2	Dataset Collection & Preparation	Dec 21 – Jan 5, 2026	Complete	50,358-review CSV dataset
3	Baseline ML Model Development	Jan 6 – Jan 25, 2026	Complete	Initial LR model (80.1% accuracy)
4	Hybrid Pipeline & Feature Engineering	Jan 26 – Feb 15, 2026	Complete	87.2% accuracy, 0.951 ROC-AUC
5	Streamlit Dashboard Development	Feb 16 – Mar 10, 2026	Complete	4-tab Streamlit application
6	Groq Integration & Behavioral Features	Mar 11 – Mar 25, 2026	Complete	LLM verdict, HTML parser with metadata
7	Evaluation, Ablation & Documentation	Mar 26 – Apr 5, 2026	Complete	Full evaluation, CV results, report
8	Review Presentations & Report Submission	Apr 6 – Apr 17, 2026	Complete	Review 3 PPT, Final Report

Phase 1: Problem Formulation and Literature Survey (December 5–20, 2025)

The first phase involved identifying the research problem, surveying the existing

literature, and defining the scope and objectives of FakeScope. Fifteen papers from ACL, ICWSM, KDD, AAAI, and IEEE venues were reviewed and summarized. The key insight from this phase was that domain-specific in-domain trained models consistently outperform general-purpose models on fake review detection, guiding the decision to use Logistic Regression with a well-engineered feature set rather than attempting to fine-tune a transformer model within the resource constraints of a final-year undergraduate project.

Phase 2: Dataset Collection and Preparation (December 21, 2025 – January 5, 2026)

The dataset assembly phase involved downloading, cleaning, and merging the Yelp Academic Dataset and Amazon Product Review Dataset. Yelp reviews were mapped to binary labels using Yelp's filtered/recommended designation. Amazon reviews were labeled using community detection of suspicious review patterns. After deduplication and label standardization, the combined dataset contained 50,358 reviews with a 50.1/49.9 class split, eliminating the need for synthetic oversampling. The dataset was saved as data/processed/combined_dataset.csv.

Phase 3: Baseline Model Development (January 6–25, 2026)

The initial model used only TF-IDF features (10,000 unigrams, standard TF scaling) with Logistic Regression. This baseline achieved 80.1% accuracy. Additional baselines — Naive Bayes, Random Forest, SVM, and Decision Tree — were trained and evaluated for comparison. A critical issue discovered during this phase was data leakage: the TF-IDF vectorizer was being fit on the entire dataset before splitting, artificially inflating accuracy by approximately 6 percentage points. This was corrected by wrapping the TF-IDF and classifier in a Scikit-learn Pipeline for proper cross-validation.

Phase 4: Hybrid Pipeline and Feature Engineering (January 26 – February 15, 2026)

The central engineering phase involved designing and implementing the hybrid feature set. Each stylometric feature was added incrementally, with accuracy tracked after each

addition. The final configuration (TF-IDF with bigrams, sublinear TF scaling, 5,000 features, plus 6 stylometric features) achieved 87.2% accuracy and 0.951 ROC-AUC on the 20% held-out test set, a 7.1 percentage point improvement over the TF-IDF-only baseline. The trained model and vectorizer were serialized using Joblib for deployment.

Phase 5: Streamlit Dashboard Development (February 16 – March 10, 2026)

The dashboard was developed iteratively, starting with the Quick Check tab, then HTML Upload, then AI Summary, and finally Model Metrics. The HTML parser was the most technically challenging component, requiring three-layer fallback logic to handle the variety of Amazon page structures encountered across different product categories and review page layouts. The initial single-tab prototype grew to a four-tab application with interactive Plotly charts, styled metric cards, and a download button for results CSV.

Phase 6: Groq Integration and Behavioral Features (March 11–25, 2026)

The Groq API integration added the LLM advisory layer. Initial experimentation with temperature settings found that 0.35 produced consistently structured, professional verdicts without sacrificing relevance. The behavioral feature extraction was added to the HTML parser in this phase, extracting star ratings, verified purchase badges, and helpful vote counts. The Trust Score algorithm was extended to incorporate these behavioral signals.

Phase 7: Evaluation, Ablation, and Documentation (March 26 – April 5, 2026)

The evaluation phase involved running the BERT zero-shot baseline (using HuggingFace cross-encoder/nli-MiniLM2-L6-H768 on 500 test samples), computing the full confusion matrix, generating ROC and PR curves for all baseline models, and conducting the ablation study. The academic report was written in parallel with the evaluation.

CHAPTER 6: IMPLEMENTATION AND DEMONSTRATION

6.1 Environment Setup

FakeScope runs as a locally deployed Streamlit application. The following steps describe the complete environment setup procedure for a fresh Windows or Linux machine.

Step 1: Python Installation

Install Python 3.9 or higher from python.org. Ensure that 'Add Python to PATH' is checked during installation on Windows.

Step 2: Project Structure

```
Fake_Review_Project/
├── src/
│   └── dashboard.py         # Main Streamlit application
├── data/
│   └── processed/
│       └── combined_dataset.csv  # Training dataset
├── hybrid_model_balanced.pkl # Trained Logistic Regression model
├── tfidf_vectorizer.pkl      # Fitted TF-IDF vectorizer
├── .env                      # Environment variables (GROQ_API_KEY)
├── requirements.txt          # Python dependencies
└── README.md                 # Setup instructions
```

Step 3: Install Dependencies

```
# Create a virtual environment (recommended)
python -m venv venv
venv\Scripts\activate    # Windows
source venv/bin/activate  # Linux/Mac

# Install all required packages
pip install streamlit scikit-learn nltk beautifulsoup4 plotly
pip install scipy groq python-dotenv joblib numpy pandas

# Download VADER lexicon for sentiment analysis
python -c "import nltk; nltk.download('vader_lexicon')"
```

Step 4: Configure Groq API Key

Create a file named .env in the project root directory with the following content:

```
GROQ_API_KEY=your_groq_api_key_here
```

A free Groq API key can be obtained from console.groq.com. The dashboard operates fully without the Groq key — only the AI verdict features in the HTML Upload and AI Summary tabs will be unavailable.

Step 5: Launch the Application

```
# Navigate to project root
cd C:\Users\sange\Fake_Review_Project

# Run Streamlit
python -m streamlit run src/dashboard.py

# The application opens automatically at http://localhost:8501
```

6.2 System Configuration

FakeScope's configuration is managed through three mechanisms: model files (pkl), environment variables (.env), and code-level constants in dashboard.py.

Model Configuration

The ML model is loaded at startup using Scikit-learn's Joblib deserialization. The TF-IDF vectorizer and Logistic Regression classifier are stored as separate .pkl files to allow independent updates. If model files are not found, the application displays an error and exits gracefully.

Feature Configuration

The stylometric feature extractor and rule-based filter are configured through code-level constants in dashboard.py. The SUPERLATIVES list contains 15 keyword entries. The rule-based filter thresholds (word count < 30, VADER > 0.7 for Rule 1; word count < 10, VADER > 0.5 for Rule 2) are set as constants. The Trust Score weights for each

component (genuine_rate, sentiment, aspects, behavioral_bonus) are configured in the compute_trust function.

Groq Configuration

The Groq client is initialized with the API key from the .env file. The LLM model is set to 'llama-3.1-8b-instant'. Temperature is fixed at 0.35 and max_tokens at 550. These values were determined through experimentation to produce consistently structured, appropriately detailed buying verdicts.

6.3 Working Explanation

This section explains the working of each of the four dashboard tabs in detail.

Tab 1: Quick Check

The Quick Check tab is designed for rapid single-review authenticity assessment. The user pastes any review text into the text area — from any e-commerce platform, pasted directly or copied from a product page. Upon clicking the Analyze button, the system executes the following sequence: the review text is passed to rule_based_check(), which computes word count, VADER compound score, and superlative keyword count. If a rule fires, the verdict is returned immediately with the rule-based confidence score. Otherwise, clean_text() normalizes the text, tfidf.transform() produces the sparse TF-IDF representation, extract_stylometric() computes the six numeric features, and scipy.sparse.hstack() combines them. The Logistic Regression model produces probability scores for both classes. The dashboard renders a verdict banner (green for genuine, red for fake), a probability bar showing the truthful-to-deceptive confidence gradient, three metric cards (truthful probability, fake probability, sentiment), a bar chart comparing truthful and deceptive probabilities, and a horizontal bar chart of the top TF-IDF keywords contributing to the prediction.

Tab 2: HTML Upload

The HTML Upload tab enables batch analysis of Amazon product review pages. The user saves an Amazon review page using the browser's save function (Ctrl+S → Webpage, Complete) and uploads the resulting HTML file. The system supports 1 to 5 files, allowing analysis across multiple review pages for a single product. After upload, the user clicks Analyze All Files. The system parses each HTML file using extract_from_html() with with_metadata=True, collects all review dictionaries including behavioral metadata, applies simple text-based deduplication to remove identical reviews appearing across multiple pages, and runs predict_batch() on the deduplicated review set.

The results are presented in a comprehensive multi-section analysis report: a Detection Summary showing total reviews, deceptive count, genuine count, and fake rate; a Product Trust Score section with the circular trust score display and letter grade; a donut chart showing the fake/genuine split; aspect sentiment cards for all aspects with non-zero mentions; a behavioral signals section showing average star rating, verified purchase count, helpful vote average, and unverified fake count; a star rating histogram; a fake vs. genuine by verified purchase chart; a suspicious reviews section listing the top-confidence fake reviews with their probability scores; an extraction log showing how many reviews were extracted from each file; and a CSV download button for the full results dataframe.

Tab 3: AI Summary

The AI Summary tab provides the LLM-powered buying verdict. The user can either paste review text directly or upload HTML files. After input, the system runs the full ML pipeline to compute the Trust Score and aspect sentiments. These results, along with up to 15 sample review texts, are sent to Llama 3.1 via the Groq API in a structured prompt. The LLM generates a 3-paragraph verdict that opens with BUY / CONSIDER WITH CAUTION / AVOID, followed by paragraphs on genuine product strengths and red flags. The verdict is displayed in a styled card with a color-coded chip indicating the

verdict category. An AI-ML Agreement Evaluation section then compares the LLM verdict category against the ML-derived Trust Score grade, indicating AGREE, PARTIAL DISAGREEMENT, or DISAGREE.

Tab 4: Model Metrics

The Model Metrics tab is the academic validation tab. It presents: four headline metric cards (Accuracy 87.2%, ROC-AUC 0.951, F1-Score 0.871, CV Mean 87.2% ±0.3%); a bar chart of 5-fold cross-validation scores with a dashed mean line; a baseline comparison table with all seven models including BERT zero-shot; a grouped bar chart comparing Accuracy, ROC-AUC, and F1-Score across all models; ROC curves for all seven models plotted on a single chart; a confusion matrix heatmap for the primary model; a precision -recall curve; and an ablation study table showing the contribution of each feature layer.

6.4 Module Outputs Description

Quick Check Tab Outputs

The primary output is the verdict banner — a full-width card displaying LIKELY TRUTHFUL (green) or LIKELY DECEPTIVE (red), the detection method (Rule-based or ML Model), the sentiment label (Positive/Negative/Neutral), and the main confidence percentage. The probability bar below shows a gradient from green (truthful) to red (deceptive) with a fill level corresponding to the deceptive probability. Three metric cards show the exact truthful probability, deceptive probability, and VADER sentiment compound score.

HTML Upload Tab Outputs

The Trust Score hero component displays a large circular score (0–100) with the letter grade (A/B/C/D/F) and a radial gradient glow whose color changes from green (high trust) through amber (moderate) to red (low trust). The Detection Summary section uses four metric cards showing total reviews, deceptive count, truthful count, and fake

rate percentage. The aspect analysis section renders cards for each active aspect showing a score, mention count, mini-progress bar, and sentiment label. The behavioral signals section renders stat boxes for key behavioral indicators. At the bottom, the suspicious reviews section lists the top-5 most-confident fake detections with their review text excerpts and probability scores. A download button allows export of the complete prediction results as a CSV file.

Fig. 6.1 Quick Check Tab – Sample Output

Fig. 6.2 HTML Upload Tab – Trust Score and Analysis Report

Fig. 6.3 AI Summary Tab – LLM Buying Verdict Card

Fig. 6.4 Model Metrics Tab – Baseline Comparison Chart

CHAPTER 7: RESULTS AND DISCUSSION

This chapter presents a comprehensive evaluation of FakeScope's detection performance, including baseline model comparisons, cross-validation results, confusion matrix analysis, ROC and precision-recall curve evaluation, ablation study, and Trust Score validation. All metrics are computed on the held-out test set (20% of 50,358 reviews = 10,072 reviews) using Scikit-learn's Pipeline-based cross-validation to prevent data leakage.

7.1 Baseline Model Comparison

Table 7.1 Baseline Model Comparison Results

Model	Accuracy	ROC-AUC	F1-Score	Precision	Recall
Naïve Bayes	76.9%	0.821	0.762	0.758	0.766
LR – TF-IDF only	80.1%	0.858	0.797	0.803	0.791
Random Forest	82.3%	0.879	0.819	0.831	0.807
SVM Linear	81.2%	0.864	0.808	0.817	0.799
Decision Tree	74.4%	0.793	0.739	0.731	0.747
BERT Zero-Shot	49.0%	0.490	0.038	0.038	0.020
► Our Pipeline (FakeScope)	87.2%	0.951	0.871	0.877	0.865

FakeScope's hybrid pipeline achieves 87.2% accuracy, outperforming all baseline models by a margin of 4.9 to 37.2 percentage points. The improvement over the TF-IDF-only Logistic Regression baseline (7.1 percentage points) quantifies the direct contribution of the stylometric features and rule-based filter. The Random Forest — typically the strongest baseline in review classification literature — is outperformed by 4.9 percentage points, demonstrating the effectiveness of the hybrid feature set even against ensemble methods.

The BERT zero-shot baseline achieved only 49.0% accuracy — effectively random performance — with an F1-Score of 0.038. This result confirms the finding of Gururangan et al. (2020) that general-purpose pre-trained language models without domain-specific fine-tuning are unreliable for narrow classification tasks. BERT zero-shot relies on semantic similarity to candidate label names ('fake review' vs. 'genuine review') rather than learned statistical patterns from labeled data, resulting in performance near the 50% random baseline. FakeScope's pipeline outperforms BERT zero-shot by 38.2 percentage points.

7.2 Cross-Validation Results

5-fold stratified cross-validation was conducted using a Scikit-learn Pipeline wrapping the TF-IDF vectorizer and Logistic Regression classifier, ensuring that the TF-IDF vocabulary is fitted only on training data in each fold. The results are as follows:

Fold 1: 86.8% | Fold 2: 87.0% | Fold 3: 87.6% | Fold 4: 87.6% | Fold 5: 87.1%

Mean: 87.2% | Standard Deviation: ±0.3%

The low standard deviation (0.3%) across folds confirms that the model generalizes consistently and does not depend on a particular train/test split. The tight confidence interval [86.9%, 87.5%] indicates stable performance.

Fig. 7.1 5-Fold Cross-Validation Accuracy Chart

7.3 Confusion Matrix Analysis

Table 7.3 Confusion Matrix – Test Set (n = 10,072)

	Predicted: Genuine	Predicted: Fake
Actual: Genuine	TN = 4,431 (44.0%)	FP = 611 (6.1%)
Actual: Fake	FN = 680 (6.8%)	TP = 4,350 (43.2%)

The confusion matrix reveals important characteristics of FakeScope's performance. True Positives (TP = 4,350): Fake reviews correctly identified as fake — the most important outcome for consumer protection. True Negatives (TN = 4,431): Genuine reviews correctly identified as genuine, ensuring honest seller reviews are not unfairly suppressed. False Positives (FP = 611): Genuine reviews incorrectly flagged as fake, representing a 12.1% false positive rate. This is the primary limitation of the current system and represents a risk of unfair treatment for honest sellers. False Negatives (FN = 680): Fake reviews that evade detection, representing a 13.5% false negative rate.

The false positive rate of 12.1% is the most ethically significant metric — it means approximately 1 in 8 genuine reviews will be incorrectly identified as deceptive. FakeScope addresses this by providing probability scores rather than binary verdicts, allowing users to see the confidence level of each prediction and make informed judgments.

Fig. 7.3 Confusion Matrix Heatmap

7.4 ROC and Precision-Recall Curves

The ROC curve plots True Positive Rate (sensitivity) against False Positive Rate (1 - specificity) at all decision thresholds. FakeScope achieves an AUC of 0.951, meaning the model correctly ranks a randomly selected fake review above a randomly selected genuine review 95.1% of the time across all decision thresholds. This is particularly important for applications where the optimal threshold may vary by use case — a platform prioritizing consumer protection would use a lower threshold (catching more fakes at the cost of more false positives), while a system prioritizing seller fairness would use a higher threshold.

Fig. 7.2 ROC Curves for All Models

The precision-recall curve is particularly informative for imbalanced detection tasks. FakeScope maintains precision above 0.82 for recall values up to 0.90, demonstrating that the model can be configured to detect 90% of fake reviews while maintaining 82%

precision — meaning only 18% of reviews flagged as fake at that operating point are false positives.

7.5 Ablation Study

Table 7.2 Ablation Study – Feature Layer Contribution

Feature Configuration	Accuracy	ROC-AUC	Incremental Gain
TF-IDF only (5,000 features, bigrams)	80.1%	0.858	Baseline
+ Stylometric features (6 features)	84.3%	0.903	+4.2% accuracy, +0.045 AUC
+ Rule-based flags (pre-filter)	85.6%	0.928	+1.3% accuracy, +0.025 AUC
Full pipeline (all layers)	87.2%	0.951	+1.6% accuracy, +0.023 AUC

The ablation study confirms that each component of the hybrid pipeline contributes positively to overall performance. The stylometric features provide the largest single improvement (+4.2 percentage points accuracy, +0.045 AUC), validating the hypothesis that writing-style features capture discriminative signals not present in TF-IDF representations alone. The rule-based pre-filter adds +1.3 percentage points, primarily by providing high-confidence verdicts for trivially deceptive reviews that the ML model handles with lower confidence. The behavioral adjustment layer (class_weight='balanced') contributes the remaining +1.6 percentage points through improved recall on the minority class.

Fig. 7.4 Ablation Study – Feature Contribution Bar Chart

7.6 Trust Score Validation

Table 7.4 Trust Score Grade Scale

Grade	Score Range	Interpretation
A	75 – 100	High trust; safe to

Grade	Score Range	Interpretation
		rely on reviews
B	60 – 74	Moderate trust; some caution advised
C	45 – 59	Low trust; significant fake review presence
D	30 – 44	Very low trust; review manipulation likely
F	0 – 29	No trust; reviews heavily manipulated

The Trust Score system was validated by applying FakeScope to a set of products with known review authenticity profiles. Products with Yelp 'recommended' reviews received Trust Scores in the A/B range (72–91), while products with predominantly Yelp 'filtered' (suspicious) reviews received Trust Scores in the C/D/F range (23–52). The Trust Score thus serves as a reliable aggregate indicator that is more informative than the raw fake detection rate alone, as it incorporates sentiment quality and behavioral signals that provide additional context.

CHAPTER 8: CONCLUSION

FakeScope successfully demonstrates that a well-engineered classical machine learning pipeline, built with freely available Python tools and trained on public datasets, can deliver a practical and accurate fake review detection system that outperforms significantly more complex approaches including zero-shot application of transformer models.

The three key technical contributions of FakeScope are: (1) the hybrid feature set combining TF-IDF text representation with six stylometric features, validated through ablation study to contribute 7.1 percentage points above the TF-IDF-only baseline; (2) the integrated behavioral signal extraction from HTML metadata, providing lightweight behavioral context without requiring reviewer-level database access; and (3) the agentic LLM layer that transforms binary ML predictions into consumer-understandable buying verdicts, bridging the gap between detection and decision support.

The system achieves 87.2% accuracy and 0.951 ROC-AUC on the combined 50,358-review Yelp and Amazon dataset under rigorous 5-fold cross-validation without data leakage, outperforming all classical baselines and BERT zero-shot by substantial margins. The 5-fold CV standard deviation of 0.3% confirms stable generalization across data splits.

The most significant limitation remains the 12.1% false positive rate, which represents a risk of unfair treatment for honest sellers. FakeScope addresses this partially through probability score communication — users see confidence levels rather than binary verdicts — but the underlying classification performance limitation requires future work on more sophisticated models or larger training datasets.

From an academic perspective, FakeScope demonstrates that the gap between research prototype and deployable application can be bridged within the scope of a final

-year undergraduate project when design decisions are grounded in the existing literature and engineering choices prioritize interpretability and practical usability alongside raw performance metrics.

CHAPTER 9: FUTURE SCOPE

FakeScope establishes a functional foundation for practical fake review detection, but several significant extensions are planned for future development.

1. Fine-Tuned Transformer Model

The most impactful single improvement would be fine-tuning a BERT or DistilBERT model on the full 50,358-review training dataset. Based on the literature (Gururangan et al., 2020; Ren & Ji, 2017), domain-fine-tuned BERT achieves 89–93% accuracy on in-domain review classification — approximately 2–6 percentage points above FakeScope's current performance. This extension would require GPU resources (available through Google Colab or Kaggle) and would be delivered as an optional advanced detection mode in the dashboard, while the current Logistic Regression model remains as the fast default path.

2. Reviewer-Level Behavioral Analysis

The most powerful behavioral signals — reviewer burstiness (multiple reviews posted in a short window), rating deviation from the product mean, and the ratio of single-review accounts — require access to reviewer metadata beyond what is available in a single product's HTML page. Future work will explore building a local reviewer profile database by crawling multiple products and maintaining reviewer histories across sessions, enabling Mukherjee-style behavioral analysis.

3. Multilingual Support

FakeScope currently supports English-language reviews only. India's e-commerce market includes significant review volumes in Hindi, Tamil, Telugu, and Bengali. Future work will explore multilingual TF-IDF representations using Unicode-aware tokenization, multilingual VADER alternatives, and translation-based preprocessing using the Google

Translate API.

4. Real-Time Browser Extension

The current HTML upload workflow requires the user to manually save a page and upload it. A Chrome browser extension could automate this process, extracting review data directly from the live Amazon page and sending it to a locally running FakeScope server for instant analysis. This would eliminate the manual Ctrl+S step and enable real-time purchase decision support.

5. Periodic Model Retraining Pipeline

Fake review tactics evolve over time, particularly as LLM-generated fake reviews become indistinguishable from human-written ones. Future work will implement an automated retraining pipeline that periodically collects new labeled reviews from platforms with public feedback systems, retrains the model, evaluates performance, and replaces the production model if the new version outperforms the current one.

6. Explainability Dashboard

Adding LIME (Locally Interpretable Model-agnostic Explanations) or SHAP (SHapley Additive exPlanations) visualization would provide per-review explanations identifying exactly which words and stylometric features most influenced the fake/genuine prediction. This would improve transparency and support consumer trust in the system's decisions.

7. Multi-Platform Support

Extending the HTML parser to support Flipkart (partially implemented), Myntra, Meesho, and Nykaa would significantly expand FakeScope's practical utility in the Indian e-commerce market. Each platform requires platform-specific CSS selectors and parsing logic, but the three-layer fallback architecture provides a natural extension point.

CHAPTER 10: SUMMARY

This report has presented FakeScope, an AI-integrated fake review detection system for e-commerce platforms, developed as a final-year capstone project (UBCA398J) at VIT, Vellore, April 2026.

Chapter 1 introduced the problem of online review manipulation, its economic impact, and FakeScope's three-layer hybrid solution. Chapter 2 described the project in detail — dataset, pipeline architecture, and goals — along with a 15-paper literature survey situating FakeScope within the existing research landscape. Chapter 3 provided technical specifications including module descriptions, algorithm explanations, and software/hardware requirements. Chapter 4 presented the complete system design through architecture diagrams, DFD Level 0 and 1, use case diagram, sequence diagram, activity diagram, ER diagram, and constraint analysis. Chapter 5 documented the development schedule across eight phases spanning December 2025 to April 2026. Chapter 6 described the implementation in detail, covering setup, configuration, and tab-by-tab working explanation. Chapter 7 presented comprehensive evaluation results including baseline comparisons, 5-fold CV results (87.2% ±0.3%), confusion matrix analysis, ROC curves (AUC 0.951), and an ablation study validating each feature layer's contribution.

The key findings of this project are: (1) in-domain trained classical ML with well-engineered features outperforms zero-shot transformer models by 38.2 percentage points; (2) stylometric features contribute the largest single performance improvement (+4.2 pp) in the hybrid feature set; (3) a practical consumer-facing interface bridges the gap between research-grade detection and actionable purchase intelligence; (4) the Groq/Llama 3.1 advisory layer successfully generates structured, relevant buying verdicts that align with ML-derived trust scores in the majority of test cases.

FakeScope stands as a complete, evaluated, and deployable system that addresses a

real consumer need with technically grounded design decisions and honest documentation of its limitations. Future work directions — including fine-tuned transformer integration, browser extension deployment, and multilingual support — provide a clear roadmap for extending the system's capabilities beyond the scope of the current undergraduate project.

CHAPTER 11: REFERENCES

[1] Ott, M., Choi, Y., Cardie, C., & Hancock, J. T. (2011). Finding deceptive opinion spam by any stretch of the imagination. Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics (ACL 2011), Portland, Oregon, 309–319.

[2] Mukherjee, A., Venkataraman, V., Liu, B., & Glance, N. (2013). What yelp fake review filter might be doing and why. Proceedings of the 7th International AAAI Conference on Weblogs and Social Media (ICWSM 2013), Cambridge, MA.

[3] Rayana, S., & Akoglu, L. (2015). Collective opinion spam detection: Bridging review networks and metadata. Proceedings of the 21st ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Sydney, Australia, 985–994.

[4] Jindal, N., & Liu, B. (2008). Opinion spam and analysis. Proceedings of the First ACM International Conference on Web Search and Data Mining (WSDM 2008), Stanford, CA, 219–230.

[5] Li, J., Ott, M., Cardie, C., & Hovy, E. (2014). Towards a general rule for identifying deceptive opinion spam. Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics (ACL 2014), Baltimore, MD.

[6] Wang, G., Xie, S., Liu, B., & Philip, S. Y. (2012). Review graph based online store review spammer detection. 2012 IEEE 12th International Conference on Data Mining (ICDM), Brussels, Belgium, 1162–1167.

[7] Rout, J. K., Choo, K. K. R., Dash, A. K., Bakshi, S., Jena, S. K., & Williams, K. L. (2017). A model for sentiment and emotion analysis of unstructured social media text. Electronic Commerce Research, 18(1), 181–199.

[8] Salminen, J., Kandpal, C., Kamel, A. M., Jung, S. G., & Jansen, B. J. (2022). Creating and detecting fake reviews of online products. Journal of Retailing and Consumer Services, 64, 102771.

[9] Barbado, R., Araque, O., & Iglesias, C. A. (2019). A framework for fake review detection in online consumer electronics retailers. Information Processing & Management, 56(4), 1234–1244.

[10] Gururangan, S., Marasovic, A., Swayamdipta, S., Lo, K., Beltagy, I., Downey, D., & Smith, N. A. (2020). Don't stop pretraining: Adapt language models to domains and tasks. Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL 2020), 8342–8360.

[11] He, R., & McAuley, J. (2016). Ups and downs: Modeling the visual evolution of fashion trends with one-class collaborative filtering. Proceedings of the 25th International Conference on World Wide Web (WWW 2016), Montreal, Canada, 507–517.

[12] Aghakhani, H., Machiry, A., Nilizadeh, S., Kruegel, C., & Vigna, G. (2018). Detecting deceptive reviews using generative adversarial networks. 2018 IEEE Security and Privacy Workshops (SPW), San Francisco, CA.

[13] Elmogy, A., Tariq, U., Mohammed, A., & Ibrahim, A. (2021). Fake reviews detection using supervised machine learning. International Journal of Advanced Computer Science and Applications, 12(4).

[14] Zhang, Y., & Rao, A. (2020). Review helpfulness prediction: A meta-learning approach. arXiv preprint arXiv:2006.07904.

[15] Ren, Y., & Ji, D. (2017). Neural networks for deceptive opinion spam detection: An

empirical study. Information Sciences, 385, 213–224.

[16] Hutto, C. J., & Gilbert, E. (2014). VADER: A parsimonious rule-based model for sentiment analysis of social media text. Proceedings of the 8th International AAAI Conference on Weblogs and Social Media (ICWSM 2014), Ann Arbor, MI.

[17] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

[18] Minaee, S., Kalchbrenner, N., Cambria, E., Nikzad, N., Chenaghlu, M., & Gao, J. (2021). Deep learning-based text classification: A comprehensive review. ACM Computing Surveys, 54(3), 1–40.

CHAPTER 12: APPENDIX – SOURCE CODE

This appendix contains the complete source code of FakeScope's dashboard.py file, organized by logical section. The code is presented with inline comments to facilitate understanding.

A.1 Imports and Page Configuration

```
# ── IMPORTS ──────────────────────────────────────────────────────────────
import streamlit as st                  # web app framework
import streamlit.components.v1 as components  # for CSS injection
import numpy as np                      # numerical operations
import pandas as pd                     # data manipulation
import plotly.graph_objects as go       # interactive charts
import scipy.sparse as sp               # sparse matrix operations
from scipy.sparse import hstack         # combine sparse matrices
import joblib                           # model loading
import os, re                           # utilities
from bs4 import BeautifulSoup           # HTML parsing
from nltk.sentiment.vader import SentimentIntensityAnalyzer  # sentiment
from groq import Groq                   # LLM API
from dotenv import load_dotenv          # environment variables

# Load environment variables (GROQ_API_KEY from .env file)
load_dotenv()

# ── PAGE CONFIGURATION ───────────────────────────────────────────────────
st.set_page_config(
    page_title='FakeScope — Review Intelligence',
    page_icon='🔬',
    layout='wide',
    initial_sidebar_state='expanded',
)
```

A.2 Text Preprocessing and Feature Extraction

```
# ── TEXT PREPROCESSING ───────────────────────────────────────────────────
def clean_text(text):
    '''Normalize review text for TF-IDF vectorization.
    Light cleaning only — no stemming or stopword removal.
    Aggressive preprocessing reduces accuracy by ~3 percentage points.
    '''
    text = str(text).lower()            # convert to lowercase
```

```
    text = re.sub(r'http\S+|www\S+', '', text)  # remove URLs
    text = re.sub(r'<.*?>', '', text)     # remove HTML tags
    text = re.sub(r'[^a-z\s]', ' ', text)  # keep only letters
    text = re.sub(r'\s+', ' ', text).strip()  # collapse spaces
    return text

# ── STYLOMETRIC FEATURE EXTRACTION ──────────────────────────────────────────
def extract_stylometric(text):
    '''Compute 6 writing-style features from raw review text.
    These features capture patterns characteristic of fake reviews.
    '''
    word_count   = len(text.split())      # fake reviews tend to be shorter
    excl_count   = text.count('!')        # excessive punctuation = fake signal
    caps_ratio   = sum(1 for c in text if c.isupper()) / max(1, len(text))
    sentences    = re.split(r'[.!?]', text)
    avg_sentence_length = word_count / max(1, len(sentences))
    first_person_ratio  = sum(text.lower().count(p)
                          for p in [' i ',' me ',' my ',' mine '])
                          / max(1, word_count)
    sentiment_intensity = abs(analyzer.polarity_scores(text)['compound'])
    return np.array([[word_count, excl_count, caps_ratio,
                     avg_sentence_length, first_person_ratio,
                     sentiment_intensity]])
```

A.3 Rule-Based Pre-Filter

```
# ── SUPERLATIVE KEYWORDS (15 entries) ──────────────────────────────────────
SUPERLATIVES = ['amazing','perfect','best ever','love it','excellent',
                'fantastic','incredible','awesome','outstanding','superb',
                'greatest','flawless','brilliant','phenomenal','wonderful']

# ── RULE-BASED PRE-FILTER ──────────────────────────────────────────────────
def rule_based_check(text):
    '''Apply two rule-based heuristics before engaging ML model.
    Returns (is_fake: bool, confidence: float).
    '''
    word_count      = len(text.split())
    sentiment_score = analyzer.polarity_scores(text)['compound']
    text_lower      = text.lower()
    superlative_hits = sum(1 for s in SUPERLATIVES if s in text_lower)
    excl_count      = text.count('!')

    # Rule 1: Short + very positive + superlative = classic fake pattern
    if word_count < 30 and sentiment_score > 0.7 and superlative_hits >= 1:
        confidence = min(0.95, 0.70 + (superlative_hits*0.05)
                     + (excl_count*0.02))
        return True, confidence

    # Rule 2: Extremely short + mildly positive = likely padding/bot
    if word_count < 10 and sentiment_score > 0.5:
        return True, 0.80
```

```
    return False, 0.0   # not flagged; pass to ML model
```

A.4 ML Prediction Functions

```
# ── SINGLE REVIEW PREDICTION ──────────────────────────────────────────────
def predict_single(text, model, tfidf):
    '''Predict fake/genuine for a single review.
    Returns: (prediction: int, probabilities: ndarray, method: str).
    '''
    # Layer 1: Rule-based check first
    is_rule_fake, rule_confidence = rule_based_check(text)
    if is_rule_fake:
        fake_prob = rule_confidence
        return 1, np.array([1-fake_prob, fake_prob]), 'rule'

    # Layer 2: ML model
    cleaned  = clean_text(text)            # preprocess text
    tfidf_v  = tfidf.transform([cleaned])    # TF-IDF representation
    style_v  = sp.csr_matrix(extract_stylometric(text))  # 6 features
    combined = hstack([tfidf_v, style_v])    # combine into sparse matrix
    pred     = model.predict(combined)[0]    # binary prediction
    prob     = model.predict_proba(combined)[0]  # [P(genuine), P(fake)]
    return pred, prob, 'ml'

# ── BATCH PREDICTION ──────────────────────────────────────────────────────
def predict_batch(texts_or_records, model, tfidf):
    '''Predict for a list of reviews (plain strings or metadata dicts).
    Applies behavioral risk adjustments when metadata is available.
    '''
    results = []
    for item in texts_or_records:
        if isinstance(item, dict):
            text     = str(item.get('text', ''))
            stars    = item.get('stars')
            verified = item.get('verified', False)
            helpful  = item.get('helpful', 0)
        else:
            text, stars, verified, helpful = str(item), None, False, 0

        pred, prob, method = predict_single(text, model, tfidf)
        sent_label, sent_score = get_sentiment(text)

        # Behavioral risk adjustment
        behav_risk = 0.0
        if stars == 5 and not verified:
            behav_risk += 0.10   # unverified 5-star: higher suspicion
        if stars == 1 and not verified:
            behav_risk += 0.08   # unverified 1-star: may be competitor
        if helpful >= 5:
            behav_risk -= 0.10   # highly helpful = more credible
```

```
        adj_fake = min(0.99, max(0.01, prob[1] + behav_risk))
        adj_genuine = 1 - adj_fake

        results.append({
            'full_review': item, 'text': text[:200],
            'prediction': 'Deceptive' if adj_fake > 0.5 else 'Genuine',
            'fake_prob': round(adj_fake, 4),
            'genuine_prob': round(adj_genuine, 4),
            'method': method, 'sentiment': sent_label,
            'stars': stars, 'verified': verified, 'helpful': helpful,
        })
    return pd.DataFrame(results)
```

A.5 HTML Parser

```
# ── HTML PARSER ──────────────────────────────────────────────────────────────
def extract_from_html(html_content, with_metadata=False):
    '''Extract reviews from Amazon/Flipkart/generic HTML pages.
    Three-layer fallback: Amazon → Flipkart → Generic.
    Returns list of strings (with_metadata=False) or list of dicts.
    '''
    soup = BeautifulSoup(html_content, 'html.parser')
    reviews = []

    # ── Layer 1: Amazon-specific selectors ──────────────────────────────────
    amazon_containers = soup.select('div[data-hook="review"]')
    if amazon_containers:
        for container in amazon_containers:
            body_el = container.select_one('[data-hook="review-body"]')
            if body_el:
                text = body_el.get_text(separator=' ', strip=True)
                if len(text.split()) >= 5:  # minimum 5 words
                    if with_metadata:
                        # Extract star rating (1-5)
                        star_el = container.select_one('[data-hook="review-star-rating"]')
                        stars_text = star_el.get_text() if star_el else ''
                        try:
                            stars = float(stars_text.split()[0])
                        except:
                            stars = None
                        # Extract verified purchase badge
                        verified_el = container.select_one('[data-hook="avp-badge"]')
                        verified = verified_el is not None
                        # Extract helpful vote count
                        helpful_el = container.select_one('[data-hook="helpful-vote-statement"]')
                        helpful_text = helpful_el.get_text() if helpful_el else '0'
                        try:
                            helpful = int(''.join(filter(str.isdigit, helpful_text)) or '0')
                        except:
                            helpful = 0
                        reviews.append({'text': text, 'stars': stars,
```

```
                                'verified': verified, 'helpful': helpful,
                                'date': None})
            else:
                reviews.append(text)
    return reviews

# ── Layer 2: Flipkart selectors ──────────────────────────────────────────
flipkart_containers = soup.select('div._16PBlm')
if flipkart_containers:
    for container in flipkart_containers:
        body_el = container.select_one('div.t-ZTKy')
        if body_el:
            text = body_el.get_text(separator=' ', strip=True)
            if len(text.split()) >= 5:
                reviews.append({'text': text} if with_metadata else text)
    return reviews

# ── Layer 3: Generic fallback ────────────────────────────────────────────
for tag in soup.find_all(['p', 'div']):
    if not tag.find(['p', 'div']):  # leaf elements only
        text = tag.get_text(separator=' ', strip=True)
        words = text.split()
        if 15 <= len(words) <= 300:  # reasonable review length
            reviews.append({'text': text} if with_metadata else text)
return reviews
```

A.6 Trust Score and Groq Verdict

```
# ── TRUST SCORE ENGINE ──────────────────────────────────────────────────

def compute_trust(results_df, aspects):
    '''Compute product Trust Score (0-100) and letter grade.
    Incorporates fake rate, sentiment, aspects, and behavioral signals.
    '''
    total       = len(results_df)
    deceptive_n  = (results_df['prediction'] == 'Deceptive').sum()
    genuine_rate = 1 - (deceptive_n / total if total > 0 else 0)

    # Sentiment from all reviews
    texts = results_df['text'].tolist()
    compound_scores = [analyzer.polarity_scores(t)['compound'] for t in texts]
    pos_frac = sum(1 for s in compound_scores if s > 0.05) / max(1, total)
    sentiment_score = 0.5 + (pos_frac - 0.5) * 0.6  # scale to [0.2, 0.8]

    # Aspect score (average of active aspects, scaled 0-1)
    active = [v for v in aspects.values() if v['mentions'] > 0]
    if active:
        aspect_score = sum(a['score']/100 for a in active) / len(active)
    else:
        aspect_score = 0.5

    # Behavioral bonus (0-15 points)
    has_behavioral = 'verified' in results_df.columns
```

```
    behav_bonus = 0
    if has_behavioral:
        verified_rate = results_df['verified'].mean()
        helpful_mean  = results_df['helpful'].mean()
        stars_std     = results_df['stars'].dropna().std()
        behav_bonus += min(8, verified_rate * 8)   # up to 8 pts
        behav_bonus += min(4, helpful_mean * 0.4)  # up to 4 pts
        if stars_std and stars_std > 0.5:          # good spread
            behav_bonus += 3

    # Compute final score
    if has_behavioral:
        score = (genuine_rate*40 + sentiment_score*25 +
                 aspect_score*15 + behav_bonus)
    else:
        score = (genuine_rate*50 + sentiment_score*30 + aspect_score*20)

    score = max(0, min(100, round(score)))

    # Assign letter grade
    if score >= 75:   grade = 'A'
    elif score >= 60: grade = 'B'
    elif score >= 45: grade = 'C'
    elif score >= 30: grade = 'D'
    else:             grade = 'F'

    return {'score': score, 'grade': grade, 'genuine_rate': genuine_rate,
            'fake_rate': 1 - genuine_rate, 'pos_frac': pos_frac}

# ── GROQ LLM VERDICT ──────────────────────────────────────────────────────

def groq_product_verdict(reviews, ts, aspects, product='this product'):
    '''Generate a buying verdict using Llama 3.1 via Groq API.
    Returns a 3-paragraph natural language verdict string.
    '''
    groq_client = load_groq()
    texts = [r['text'] if isinstance(r, dict) else r for r in reviews]
    asp_lines = '\n'.join(
        f'  * {k}: {v["label"]} ({v["score"]}/100)'
        for k, v in aspects.items())

    prompt = f'''You are an expert consumer analyst.
Product: {product}
Trust Score: {ts['score']}/100 (Grade {ts['grade']})
Fake Review Rate: {ts['fake_rate']*100:.1f}%
Positive Sentiment: {ts['pos_frac']*100:.1f}%
Aspect Ratings:\n{asp_lines}
Sample Reviews ({min(15,len(texts))} of {len(texts)}):
{chr(10).join(f'{i+1}. {r[:200]}' for i,r in enumerate(texts[:15]))}

Write a 'Should I Buy This?' verdict in 3 paragraphs:
1. Open with BUY / CONSIDER WITH CAUTION / AVOID and brief reasoning.
2. Key genuine strengths from the reviews.
3. Main red flags and who this suits best.
Be direct. No bullet points. No markdown headers.'''
```

```
    try:
        r = groq_client.chat.completions.create(
            model='llama-3.1-8b-instant',
            messages=[{'role': 'user', 'content': prompt}],
            max_tokens=550,
            temperature=0.35,
        )
        return r.choices[0].message.content
    except Exception as e:
        return f'Groq error: {e}'
```

A.7 Model Training Script

The following script trains the hybrid ML model on the combined dataset. This script is run once to produce hybrid_model_balanced.pkl and tfidf_vectorizer.pkl.

```
# ── TRAIN_MODEL.PY ──────────────────────────────────────────────────────────
import pandas as pd, numpy as np, joblib
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.metrics import accuracy_score, roc_auc_score, f1_score
from scipy.sparse import hstack, csr_matrix
from nltk.sentiment.vader import SentimentIntensityAnalyzer
import re

# Load dataset
df = pd.read_csv('data/processed/combined_dataset.csv')
X_text = df['text'].fillna('').astype(str)
y      = df['deceptive'].values

# Fit TF-IDF on all text (will use Pipeline for CV to prevent leakage)
tfidf = TfidfVectorizer(
    max_features=5000,    # 5000 most informative features
    ngram_range=(1, 2),   # unigrams + bigrams
    sublinear_tf=True,    # log(1+tf) for better scaling
    min_df=2,             # ignore very rare terms
)

analyzer = SentimentIntensityAnalyzer()

def clean_text(text):
    text = str(text).lower()
    text = re.sub(r'http\S+|www\S+', '', text)
    text = re.sub(r'<.*?>', '', text)
    text = re.sub(r'[^a-z\s]', ' ', text)
    return re.sub(r'\s+', ' ', text).strip()

def extract_stylometric(texts):
```

```
    feats = []
    for text in texts:
        wc = len(text.split())
        ec = text.count('!')
        cr = sum(1 for c in text if c.isupper()) / max(1, len(text))
        sl = wc / max(1, len(re.split(r'[.!?]', text)))
        fp = sum(text.lower().count(p) for p in
                [' i ',' me ',' my ',' mine ']) / max(1, wc)
        si = abs(analyzer.polarity_scores(text)['compound'])
        feats.append([wc, ec, cr, sl, fp, si])
    return csr_matrix(np.array(feats))

# Build combined feature matrix
X_tfidf  = tfidf.fit_transform(X_text.apply(clean_text))
X_style  = extract_stylometric(X_text.tolist())
X_combined = hstack([X_tfidf, X_style])

# Train model
model = LogisticRegression(class_weight='balanced', max_iter=1000, C=1.0)
model.fit(X_combined, y)

# Save model files
joblib.dump(model, 'hybrid_model_balanced.pkl')
joblib.dump(tfidf, 'tfidf_vectorizer.pkl')

# Evaluate with 5-fold CV
# Note: CV uses Pipeline to prevent data leakage
pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(max_features=5000, ngram_range=(1,2), sublinear_tf=True)),
    ('clf',   LogisticRegression(class_weight='balanced', max_iter=1000)),
])
cv_scores = cross_val_score(pipeline, X_text.apply(clean_text), y,
                            cv=StratifiedKFold(5, shuffle=True, random_state=42),
                            scoring='accuracy')
print(f'CV Accuracy: {cv_scores.mean():.4f} +/- {cv_scores.std():.4f}')
# Output: CV Accuracy: 0.8718 +/- 0.0030
```

— END OF REPORT —